‘대학 축제 통합 플랫폼’ 설계 보고서

실무중심산학협력프로젝트1(캡스톤디자인-SW) 3분반

팀 명 **단짠 (DANZZAN)**

이 름 강하늘, 박주희, 박지우, 조하은

목차

목차 2

I. 프로젝트 개요 4

1. 프로젝트 주제 및 선정 배경 4

2. 프로젝트 목표 4

3. 기대 효과 5

II. 문제 정의 및 서비스 분석 6

1. 기존 운영 방식 분석 및 문제점 6

2. 유사 서비스 분석 7

3. 서비스 차별성 8

III. 프로젝트 설계 9

1. 전체 서비스 구조 9

2. 주요 기능 및 요구사항 설계 11

3. 사용자 플로우 설계 12

4. 화면 설계 14

5. 데이터베이스 설계 18

6. 시스템 아키텍처 설계 19

7. 서버 구성 및 배포 구조 설계 21

8. 데이터 형식 및 API 계약 설계 21

9. 티켓팅 정합성 설계 24

10. Docker Compose 기반 인프라 설계 24

11. 인증 및 보안 설계 25

IV. 구현 기술 및 개발 환경 26

1. 주요 기술 스택 26

2. 티켓팅 대기열 구현 방식 27
3. 개발 및 협업 환경 28
1. 커뮤니케이션 체계 29
2. 일정 및 이슈 관리 방식 29
3. 브랜치 전략 및 코드 리뷰 프로세스 31
4. 데일리스크럼 운영 33
5. 모니터링 및 부하 테스트 환경 33
6. 스프린트 회고 및 협업 개선 33
7. 개발 프로세스 34
V. 진행 상황 및 향후 계획 37
1. 현재 진행 상황 37
2. 배포 이후 운영 계획 37
3. 향후 개선 방향 38

I. 프로젝트 개요

1. 프로젝트 주제 및 선정 배경

본 프로젝트의 주제는 대학 축제 통합 플랫폼 DANZZAN 의 개발이다. DANZZAN 은 대학 축제 기간 동안 학생들이 필요로 하는 축제 정보를 하나의 웹 서비스에서 통합적으로 제공하고, 티켓팅과 내 티켓 확인 기능까지 연계하여 지원하는 서비스로 정의하였다. 사용자는 홈 화면, 타임테이블, 부스맵, 공지사항, 티켓팅 기능을 통해 축제 참여에 필요한 정보를 한 곳에서 확인할 수 있으며, 운영진은 관리자 페이지를 통해 공지, 광고 배너, 지도 정보, 티켓 배부 현황 등을 관리할 수 있도록 구성하였다.

대학 축제는 짧은 기간 동안 많은 학생이 동시에 참여하는 행사이므로 정확한 정보 전달과 효율적인 운영이 중요하다. 그러나 기존 축제 운영 방식에서는 공지사항, 공연 일정, 부스 위치, 티켓 예매 정보 등이 인스타그램, 에브리타임, 외부 폼, 외부 티켓팅 서비스 등 여러 채널에 분산되어 제공되는 경우가 많았다. 그 결과 학생들은 필요한 정보를 찾기 위해 여러 플랫폼을 이동해야 했으며, 최신 정보를 스스로 선별하여 확인해야 하는 불편을 겪었다.

운영진의 관점에서도 동일한 공지를 여러 채널에 반복 게시해야 하며, 티켓 예매 현황과 현장 배부 현황을 일관되게 관리하기 어렵다는 문제가 존재하였다. 특히 축제 현장에서는 짧은 시간 내에 많은 학생의 티켓 보유 여부와 신분을 확인해야 하므로, 운영 절차가 복잡할수록 대기 혼잡과 업무 부담이 증가할 가능성이 높다.

이에 본 프로젝트는 축제 정보 제공, 티켓팅, 현장 운영 관리 기능을 하나의 서비스로 통합하여 학생에게는 편리한 축제 이용 경험을 제공하고 운영진에게는 효율적인 관리 환경을 제공하는 방향으로 기획하였다. 또한 단순한 학습용 프로젝트에 그치지 않고 실제 축제 운영에 적용 가능한 웹 서비스 개발을 목표로 설정하였으며, 향후 다양한 교내 행사로 확장 가능한 플랫폼을 지향하도록 설계하였다.

2. 프로젝트 목표

본 프로젝트의 목표는 크게 네 가지로 정리하였다.

첫째, 축제 정보의 통합 제공이다. 사용자는 홈 화면, 타임테이블, 부스맵, 공지사항 기능을 통해 **축제 관련 정보를 한 곳에서 확인**할 수 있도록 하였다. 이를 통해 여러 채널을 오가며 정보를 탐색해야 하는 불편을 줄이고 축제 정보 접근성을 높이고자 하였다.

둘째, **학생 참여 기능을 포함한 티켓팅 서비스 제공**이다. DANZZAN은 단순한 축제 정보 제공 서비스에 그치지 않고 학생 인증 기반의 티켓 예매 기능을 포함하도록 설계하였다. 사용자는 서비스 내에서 티켓팅에 참여하고 예매 완료 후 내 티켓 화면에서 예매 내역을 확인할 수 있도록 하였으며, 이를 통해 축제 정보 확인부터 티켓 예매, 티켓 확인까지 이어지는 흐름을 하나의 서비스 내에서 제공하고자 하였다.

셋째, **실제 축제 운영을 지원하는 관리자 기능 제공**이다. 관리자 페이지에서는 공지사항 작성 및 수정, 광고 배너 관리, 티켓 조회 및 팔찌 배부 처리 등을 수행할 수 있도록 구성하였다. 특히 티켓 배부 현황을 관리자가 직접 확인하고 처리할 수 있도록 하여 실제 축제 현장에서 운영진의 업무 부담을 줄이는 것을 목표로 설정하였다.

넷째, **실제 배포와 향후 확장을 고려한 플랫폼 구축**이다. 본 프로젝트는 단순한 학습용 프로젝트가 아니라 실제 사용 가능한 웹 서비스 구현을 목표로 설정하였다. 축제 기간 동안 실제 사용자를 대상으로 운영 가능한 서비스를 개발하고, 향후 체육대회, 학생회 행사, 설명회, 특강 등 다양한 교내 행사로 확장 가능한 구조를 지향하도록 설계하였다.

3. 기대 효과

본 프로젝트를 통해 기대되는 효과는 다음과 같다.

1) 축제 정보 접근성 향상

축제 정보가 하나의 서비스에 통합될 경우 학생들은 여러 채널을 이동하지 않고도 필요한 정보를 확인할 수 있다. 이를 통해 공지 누락, 일정 착오, 위치 탐색의 어려움 등을 줄일 수 있을 것으로 기대된다.

2) 운영 효율성 향상 및 현장 혼선 완화

운영진은 관리자 페이지를 통해 공지, 광고 배너, 티켓 배부 현황을 관리할 수 있다. 또한 티켓 예매와 현장 배부 처리가 하나의 시스템 내에서 연계되므로 기존과 같이 명단, QR, 수동 확인 절차가 분산되는 문제를 완화할 수 있다. 특히 팔찌 배부 현황을 한 화면에서 확인하고 처리할 수 있도록 함으로써 현장 스태프의 업무 부담을 경감하는 효과를 기대할 수 있다.

3) 사용자 데이터 기반의 지속 가능한 플랫폼 구축

DANZZAN 은 자체 회원 시스템과 티켓팅 기능을 포함하므로 축제 이후에도 사용자 기반을 유지하면서 다른 교내 행사로 확장할 가능성을 가진다. 이는 매년 새로운 서비스를 개별적으로 구축하는

방식보다 효율적이며, 학교 행사 운영과 관련된 디지털 자산을 축적하는 방향으로 발전할 수 있음을 의미한다.

4) 향후 교내 통합 행사 플랫폼으로의 확장 가능성

현재는 대학 축제를 중심으로 서비스를 설계하였으나, 이후에는 체육대회, 학생회 행사, 특강, 설명회 등 다양한 교내 행사에도 동일한 플랫폼 구조를 적용할 수 있도록 확장성을 고려하였다. 다시 말해 DANZZAN 은 일회성 축제 애플리케이션을 넘어 교내 행사를 통합적으로 관리하고 안내하는 플랫폼으로 발전할 가능성을 가진다.

II. 문제 정의 및 서비스 분석

1. 기존 운영 방식 분석 및 문제점

기존 대학 축제 운영 방식은 정보 제공, 티켓 예매, 현장 운영 관리가 상호 분리된 구조를 가진다. 공지사항은 인스타그램이나 에브리타임과 같은 외부 채널을 통해 전달되고, 티켓 예매는 네이버 폼, 구글폼 또는 외부 티켓팅 서비스로 진행되는 경우가 많다. 또한 현장에서는 학생증 확인, QR 확인, 명단 확인, 팔찌 배부 등의 절차가 별도로 이루어져 운영 흐름이 복잡해지는 경향이 있다.

1) 사용자 관점의 문제점

사용자는 축제에 필요한 정보를 확인하기 위해 여러 채널을 이동해야 한다. 공지사항, 티켓 예매 링크, 공연 일정, 부스 위치가 서로 다른 곳에 게시될 경우 필요한 정보를 찾는 데 추가적인 시간이 소요된다. 또한 우천, 공연 지연, 장소 변경과 같은 변동 사항이 발생했을 때 모든 채널에 동일한 정보가 동시에 반영되지 않으면, 사용자는 어느 정보가 최신 정보인지 판단하기 어렵다.

이러한 구조는 정보 피로도를 높이고 학생별로 주로 사용하는 채널에 따라 정보 접근성 차이를 발생시킨다. 인스타그램을 주로 확인하는 학생과 에브리타임을 주로 확인하는 학생, 또는 두 채널을 모두 자주 확인하지 않는 학생 사이에 정보 격차가 생길 수 있다.

2) 운영진 관점의 문제점

운영진은 동일한 공지사항을 여러 채널에 반복해서 게시해야 하며 각 채널에서 발생하는 댓글, DM, 채팅 문의에 개별적으로 대응해야 한다. 이로 인해 운영 부담이 증가하고 채널별로 안내 내용이 다르게 전달될 가능성도 존재한다.

또한 티켓 예매 정보와 현장 배부 정보가 분리되어 있으면 누가 예매를 완료했는지, 누가 실제로 팔찌를 수령했는지, 어느 날짜의 배부가 얼마나 진행되었는지를 실시간으로 파악하기 어렵다. 문제 상황이 발생했을 때도 관련 기록이 여러 시스템에 흩어져 있으면 사실 관계를 빠르게 확인하기 어렵다.

3) 현장 티켓 운영 관점의 문제점

축제 현장에서는 짧은 시간 안에 많은 학생을 확인하고 입장 또는 팔찌 배부를 처리해야 한다. 기존 방식처럼 QR 확인, 학생증 확인, 명단 확인, 팔찌 배부가 단계별로 분리되어 있으면 처리 시간이 길어지고 현장 대기열이 증가할 수 있다.

또한 QR 스캔이나 외부 인증 절차에 의존하는 경우 네트워크 상태, 기기 인식 문제, 인증 지연 등의 변수가 발생할 수 있다. 따라서 현장에서는 학생 정보와 티켓 상태를 빠르게 조회하고 지급 여부를 즉시 처리할 수 있는 간소화된 운영 방식이 필요하다.

2. 유사 서비스 분석

유사 서비스는 크게 기존 단국대학교 축제 운영 방식, 타 대학 축제 서비스, 외부 티켓팅 및 QR 기반 운영 서비스로 구분하여 분석할 수 있다. 기존 서비스들은 축제 정보 제공이나 티켓팅 기능 중 일부를 제공하지만 축제 정보 확인부터 티켓 예매, 내 티켓 확인, 현장 운영 관리까지 하나의 흐름으로 통합하는 데에는 한계가 있다.

기존 단국대학교 축제 운영 방식은 인스타그램, 에브리타임, 외부 폼, 외부 티켓팅 서비스 등 다양한 채널을 활용하는 방식에 가깝다. 이러한 방식은 이미 학생들이 익숙하게 사용하는 채널을 활용할 수 있다는 장점이 있지만 정보와 기능이 여러 곳에 흩어져 있어 사용자가 직접 이동해야 한다는 한계가 있다. 특히 공지는 특정 SNS 에서 확인하고 티켓 예매는 별도의 링크에서 진행하며 현장에서는 다시 학생증이나 QR 을 통해 확인하는 방식으로 이어질 경우 사용자 경험이 분절될 수 있다.

타 대학 축제 서비스의 경우 축제 일정, 공지, 이벤트 안내 등 정보 제공 중심의 서비스가 존재한다. 이러한 서비스는 학생들이 축제 정보를 확인하는 데 도움을 줄 수 있지만 티켓팅이나 현장 배부 관리까지 하나의 시스템 안에서 처리하지 않는 경우에는 실제 운영 효율성 측면에서 한계가 있다. 즉, 앱 또는 웹 서비스에서 축제 정보를 확인하더라도 티켓 예매나 현장 확인은 다른 서비스나 오프라인 절차에 의존하게 된다.

외부 티켓팅 또는 QR 기반 서비스는 예매와 입장 확인 기능을 제공할 수 있다는 장점이 있다. 그러나 대학 축제의 전체 운영 관점에서 보면 공지사항, 공연 일정, 부스맵, 광고 배너, 관리자 운영 기능과 자연스럽게 연결되지 않을 수 있다. 또한 QR 기반 확인 방식은 현장에서 빠르게 처리할 수 있다는 장점이

있지만 QR 스캔, 본인 확인, 팔찌 배부 등 절차가 분리될 경우 운영 인력이 많이 필요하고 대기 시간이 길어질 수 있다.

유사 서비스 분석 결과는 다음과 같이 정리할 수 있다.

구분	특징	한계
기존 단국대 운영 방식	SNS, 커뮤니티, 외부 폼 등을 활용하여 공지와 티켓 안내 진행	정보와 기능이 여러 채널에 분산되어 사용자 경험이 끊어질 수 있음
타 대학 축제 서비스	축제 일정, 공지, 이벤트 등 정보 제공 중심	티켓팅과 현장 운영 관리까지 통합되지 않는 경우가 있음
외부 티켓팅 서비스	예매, QR 확인 등 티켓 중심 기능 제공	축제 정보, 공지, 부스맵, 관리자 운영 기능과의 연결이 제한적일 수 있음
QR 기반 현장 확인	모바일 화면을 활용한 빠른 확인 가능	QR 인식, 본인 확인, 배부 처리 절차가 분리되면 현장 운영이 복잡해질 수 있음

따라서 본 프로젝트는 기존 서비스들의 장점인 정보 제공 기능과 티켓팅 기능을 반영하되, 이를 하나의 서비스 내에서 유기적으로 연결하고 관리자 운영 기능까지 포함하는 방향으로 차별화하였다.

3. 서비스 차별성

DANZZAN 의 가장 큰 차별성은 축제 정보 제공, 티켓팅, 현장 운영 관리 기능을 하나의 서비스 내에 통합하였다는 점이다. 기존 방식에서는 공지 확인, 티켓 예매, 현장 확인이 각각 다른 채널이나 절차에서 이루어질 수 있었으나, DANZZAN 은 사용자가 축제 정보를 확인하고 티켓팅에 참여한 뒤 내 티켓을 확인하는 흐름을 하나의 웹 서비스 내에서 일관되게 제공하도록 설계하였다.

첫째, DANZZAN 은 끊김 없는 사용자 경험을 제공하도록 설계하였다. 사용자는 홈 화면에서 축제의 주요 안내를 확인하고, 타임테이블을 통해 공연 일정을 확인하며, 부스맵을 통해 현장 위치 정보를 탐색할 수 있도록 하였다. 또한 공지사항을 통해 변경 사항이나 긴급 안내를 확인하고 티켓팅 기능을 통해 예매까지 진행할 수 있도록 구성함으로써, 축제 참여에 필요한 주요 기능이 하나의 서비스 내에서 연결되도록 하였다.

둘째, 관리자 페이지를 통해 실제 운영을 지원하도록 설계하였다. 운영진은 공지사항과 긴급 공지를 등록하고 광고 배너를 관리하며 지도와 부스 정보를 수정할 수 있도록 구성하였다. 또한 티켓 예매 현황과 현장 발급 상태를 확인하고 학번을 기준으로 티켓 보유 여부를 조회하여 팔찌 배부 여부를 처리할 수 있도록 함으로써, 여러 채널과 문서를 오가며 관리해야 하는 부담을 줄이고 현장 운영 상황을 보다 체계적으로 관리할 수 있도록 하였다.

셋째, 티켓팅 기능에 대기열 구조를 적용하여 동시 접속 상황을 고려하였다. 축제 티켓팅은 특정 시간에 많은 사용자가 동시에 접속하는 특성이 있으므로 단순한 요청 처리 방식만으로는 안정적인 운영을 보장하기 어렵다. 이에 DANZZAN 은 대기열을 통해 사용자의 진입 순서를 관리하고 예매 가능 상태에 도달한 사용자만 예매를 진행하도록 설계하여 서버 부하와 중복 예매 문제를 완화하고자 하였다.

넷째, 현장 운영 절차를 간소화할 수 있도록 설계하였다. 기존 방식처럼 QR 스캔, 학생증 확인, 명단 확인, 팔찌 배부가 분리되어 있을 경우 현장 처리 흐름이 복잡해질 수 있다. 이에 DANZZAN 은 관리자 페이지에서 학생 정보와 티켓 상태를 함께 조회하고 발급 처리를 수행할 수 있도록 구성하여, 현장 스태프가 한 화면에서 필요한 정보를 확인하고 처리할 수 있는 구조를 제공하였다.

다섯째, 향후 교내 행사 플랫폼으로의 확장 가능성을 고려하였다. DANZZAN 은 회원 기반 서비스와 관리자 기능을 포함하고 있기 때문에 일회성 축제 안내 페이지에 머무르지 않는다. 축제 이후에도 체육대회, 학생회 행사, 설명회, 특강 등 다양한 교내 행사에 적용할 수 있으며, 학교 행사 운영을 위한 통합 플랫폼으로 발전할 가능성을 가진다.

차별화 요소	설명
끊김 없는 사용 경험	공지, 지도, 타임테이블, 티켓팅, 내 티켓 확인을 하나의 서비스에서 제공
운영진 전용 관리 기능	공지, 광고, 지도, 티켓 배부 현황을 관리자 페이지에서 관리
대기열 기반 티켓팅	동시 접속 상황을 고려한 순서 관리 및 예매 흐름 제공
현장 운영 절차 간소화	학생 번호 기준 조회와 티켓 상태 확인을 통해 팔찌 배부 처리 지원
플랫폼 확장성	회원 기반 구조를 통해 향후 다른 교내 행사로 확장 가능

결과적으로 DANZZAN 은 단순히 축제 정보를 제공하는 서비스가 아니라 축제 정보 탐색, 티켓 예매, 내 티켓 확인, 현장 발급 처리, 관리자 운영 관리까지 연결하는 대학 축제 통합 플랫폼으로서 기존 방식과의 차별성을 확보하였다.

III. 프로젝트 설계

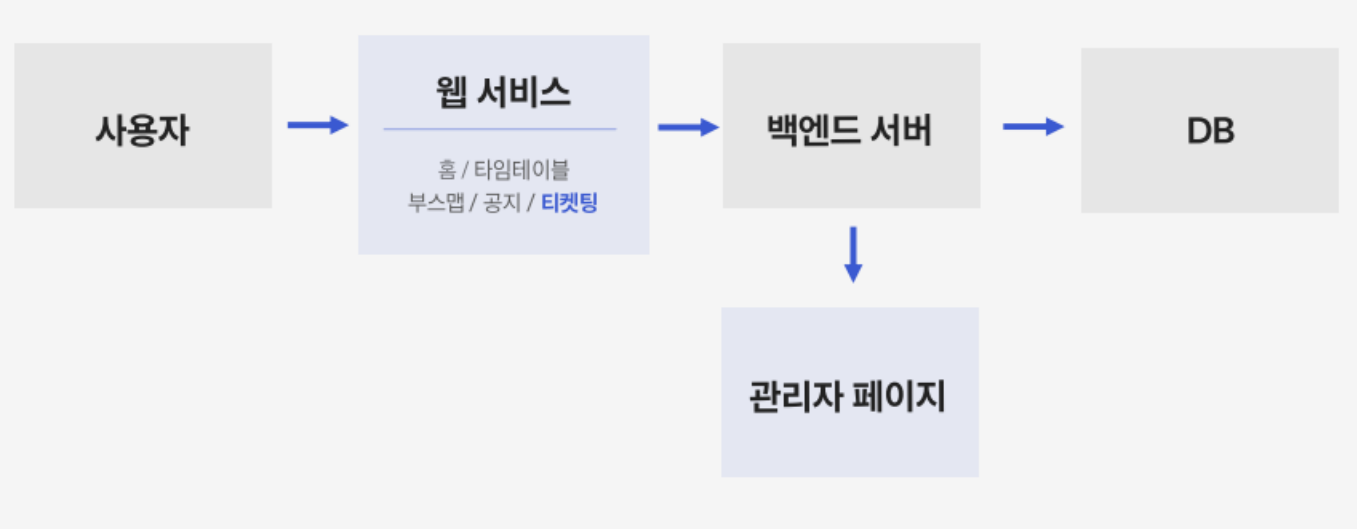
1. 전체 서비스 구조

DANZZAN 은 대학 축제 정보를 통합적으로 제공하고 티켓팅과 현장 운영 관리까지 지원하는 웹 기반 서비스이다. 전체 서비스는 크게 사용자 서비스, 티켓팅 서비스, 관리자 서비스로 구성된다.

사용자 서비스는 축제 참여자가 홈 화면, 타임테이블, 부스맵, 공지사항을 통해 축제 정보를 확인하는 영역이다. 사용자는 축제 대표 이미지, 라인업, 공연 일정, 주점 및 부스 위치, 긴급 공지 등을 확인할 수 있다.

티켓팅 서비스는 학생 인증을 완료한 사용자가 공연 또는 행사 티켓 예매에 참여하는 영역이다. 사용자는 로그인 후 티켓팅 화면에 접근하고, 대기열을 거쳐 예매 가능 상태가 되면 티켓을 예매한다. 예매 완료 후에는 내 티켓 화면에서 예매 내역과 티켓 상태를 확인할 수 있다.

관리자 서비스는 축제 운영진이 서비스 콘텐츠와 현장 운영을 관리하는 영역이다. 관리자는 공지사항, 긴급 공지, 광고 배너, 부스 및 주점 정보, 티켓팅 이벤트를 관리할 수 있으며, 현장에서 학생의 티켓 보유 여부를 조회하고 팔찌 배부 여부를 처리할 수 있다.



2. 주요 기능 및 요구사항 설계

DANZZAN 의 기능 요구사항은 사용자 기능과 관리자 기능으로 구분된다. 사용자 기능은 축제 참여자가 서비스를 이용하는 흐름을 중심으로 구성되며 관리자 기능은 운영진이 축제 정보를 관리하고 현장 운영을 처리하는 흐름을 중심으로 구성된다.

사용자 기능 요구사항

사용자 기능은 홈, 타임테이블, 부스맵, 공지사항, 회원 및 인증, 티켓팅 기능으로 구성된다.

번호	대상	분류	기능명	기능 설명	비고
1	사용자	홈	홈 이미지 조회	홈 상단 배너 및 대표 이미지를 순서대로 표시	관리자 등록 이미지
2			라인업 이미지 조회	라인업 소개 이미지를 홈 화면에서 조회	이미지형 콘텐츠
3			긴급 공지 표시	활성화된 긴급 공지가 있을 경우 홈 화면 상단에 노출	없으면 미노출
4			광고 조회	위치별 활성 광고 1건을 조회하여 화면에 노출	홈 하단, 내 티켓 화면 등
5			광고 목록 조회	현재 활성화된 광고 목록을 조회	캐러셀 또는 리스트 활용 가능
6		타임테이블	날짜별 공연 조회	선택한 날짜의 공연 타임테이블을 조회	날짜 기준 조회
7			공연 상세 정보 확인	공연 시간, 무대, 출연자명, 출연자 소개, 이미지 등을 확인	타임테이블 항목별 정보
8			콘텐츠 이미지 조회	타임테이블 화면에서 사용하는 콘텐츠 이미지 목록을 조회	미리보기 및 상세 이미지 포함
9		부스맵	전체 부스맵 조회	운영 날짜 기준 단과대 마커와 부스 마커 정보를 지도에서 조회	활성 운영 날짜 기준 가능
10			부스 요약 조회	지도에서 선택한 부스의 이름, 설명, 이미지, 운영시간 등 요약 정보를 조회	체험부스, 푸드트럭, 편의시설
11			주점 목록 조회	운영 날짜 기준 전체 주점 목록을 조회	학과 및 단과대 정보 포함
12			주점 상세 조회	특정 주점의 소개, 설명, 인스타그램, 이미지 목록, 운영시간을 조회	상세 페이지 또는 하단 패널
13			운영 날짜별 정보 조회	조회 날짜에 따라 부스 및 주점 운영시간 정보를 다르게 확인	날짜 파라미터 지원
14		공지	공지사항 목록 조회	활성화된 공지사항 목록을 페이지 단위로 조회	최신순 기본
15			공지사항 검색	키워드 기준으로 공지사항을 검색	제목 또는 내용 기반 활용
16			공지사항 상세 조회	특정 공지사항의 제목, 본문, 이미지, 작성자, 작성일 등을 확인	상세 페이지
17		회원/인증	단국대 학생 인증	단국대 포털 정보로 학생 인증을 수행하고 회원가입용 토큰을 발급	회원가입 1단계
18			학생 인증 정보 조회	발급된 회원가입 토큰으로 인증된 학생 정보를 조회	이름, 학과 등
19			전화번호 인증 세션 생성	회원가입 과정에서 전화번호 인증 세션을 생성하고 인증 절차를 시작	인증코드 검증용
20			전화번호 인증 검증	입력한 전화번호가 인증 조건을 만족하는지 검증	회원가입 2단계
21			전화번호 인증 상태 조회	인증 세션의 현재 상태를 조회	대기, 성공, 만료 등
22			회원가입 완료	회원가입 토큰과 사용자 정보를 이용해 가입을 완료	비밀번호 설정 포함
23			로그인	학번과 비밀번호로 로그인	JWT 기반
24			토큰 재발급	Refresh Token으로 Access Token을 재발급	로그인 유지
25			로그아웃	로그아웃 처리 후 기존 토큰을 무효화	토큰 버전 사용
26			내 정보 조회	현재 로그인한 사용자의 정보를 조회	마이페이지
27			비밀번호 재설정 요청	학번 기반으로 비밀번호 재설정 인증코드를 요청	메일 또는 인증 절차 포함
28			비밀번호 재설정 코드 검증	전달받은 인증코드를 검증하고 재설정 토큰을 발급	다음 단계 진행용
29			비밀번호 재설정	검증된 토큰으로 새 비밀번호로 변경	계정 복구
30		티켓팅	티켓팅 가능 이벤트 조회	예매 가능한 공연 및 이벤트 목록을 조회	날짜별 행사
31			대기열 진입	특정 이벤트 예매를 위해 대기열에 진입	중복 진입 방지
32			대기열 이탈	사용자가 직접 대기열에서 이탈	대기 취소
33			대기열 상태 조회	현재 대기 순번, 앞 대기 수, 예상 대기시간 등을 조회	폴링 방식
34			예매 화면 활성화	READY 상태에서 실제 예매 화면 진입을 활성화	유의사항 확인 후 진입
35			티켓 예매	ACTIVE 상태에서 티켓을 최종 예매	정원 차감 처리
36			내 티켓 조회	사용자가 예매한 티켓 목록을 조회	마이티켓

관리자 기능 요구사항

관리자 기능은 관리자 인증, 지도 관리, 광고 관리, 공지 관리, 이벤트 관리, 티켓 관리 기능으로 구성된다.

번호	대상	분류	기능명	기능 설명	비고
1	관리자	관리자 인증	관리자 로그인	관리자 계정으로 로그인하여 관리자 기능에 접근	별도 관리자 인증
2			관리자 토큰 재발급	쿠키 기반 Refresh Token으로 토큰 재발급	관리자 세션 유지
3			관리자 로그아웃	관리자 로그아웃 및 Refresh Token 무효화	보안 처리
4		지도 관리	관리자 지도 조회	운영 날짜 기준 단과대 및 부스 위치 현황을 조회	배치 관리 화면
5			단과대 위치 수정	지도 상 단과대 마커 위치를 수정	드래그 또는 좌표 입력 활용 가능
6			부스 위치 수정	지도 상 부스 마커 위치를 수정	운영 배치 관리
7			부스 위치 삭제	부스의 지도 좌표를 제거하여 미배치 상태로 전환	재배치 가능
8		광고 관리	광고 전체 조회	등록된 광고 목록을 전체 조회	삭제 제외 목록
9			광고 등록/교체	위치별 광고를 새로 등록하거나 기존 광고를 교체	placement 기준
10			광고 이미지 업로드 URL 발급	광고 이미지 업로드용 Presigned URL을 발급	S3 업로드
11			광고 활성화/비활성화	광고 노출 여부를 제어	위치별 on/off
12			광고 삭제	광고를 소프트 삭제 처리	사용자 화면에서 미노출
13		공지 관리	긴급 공지 조회	현재 긴급 공지 상태를 조회	단일 공지 관리
14			긴급 공지 수정	긴급 공지 메시지와 활성 여부를 수정	홈 긴급공지 반영
15			공지사항 목록 조회	공지 목록을 상태, 카테고리, 검색 조건으로 조회	관리자용 목록
16			공지사항 등록	제목, 내용, 이미지 등을 포함한 공지를 등록	일반, 이벤트, 긴급 여부
17			공지사항 수정	기존 공지 내용을 수정	이미지 포함
18			공지사항 삭제	공지를 소프트 삭제 처리	복구 가능
19			공지사항 복구	삭제된 공지를 다시 활성화	isActive 복구
20			공지 순서 변경	공지 displayOrder를 변경하여 노출 순서를 조정	고정 공지 정렬용
21			공지 이미지 업로드	공지에 사용할 이미지를 업로드	S3 업로드
22			공지 이미지 업로드 URL 발급	공지 이미지 업로드용 Presigned URL을 발급	클라이언트 직접 업로드
23		이벤트 관리	이벤트 목록 조회	관리자 화면에서 티켓팅 이벤트 목록을 조회	상태 및 정원 확인
24			이벤트 수동 오픈	READY 상태 이벤트를 수동으로 OPEN 처리	티켓팅 시작
25			이벤트 통계 조회	이벤트별 총 정원, 예매 수, 발급 수, 잔여 수량을 조회	운영 지표 확인
26		티켓 관리	학생 번호 기준 티켓 조회	특정 학생의 이벤트 티켓 보유 현황을 조회	현장 확인용
27			티켓 발급 처리	예매 완료 티켓을 현장 발급 상태로 변경	발급 관리자 기록
28			티켓 발급 취소	발급된 티켓을 다시 미발급 상태로 되돌림	오처리 복구

3. 사용자 플로우 설계

사용자 플로우는 일반 사용자 플로우, 티켓팅 사용자 플로우, 관리자 플로우로 구분하여 설계하였다. 이는 축제 정보를 단순히 조회하는 사용자와 티켓팅에 참여하는 사용자, 그리고 실제 운영을 담당하는 관리자의 사용 목적이 서로 다르기 때문이다.

일반 사용자 플로우

일반 사용자는 홈 화면에서 축제 정보를 확인한 뒤 하단 내비게이션을 통해 타임테이블, 부스맵, 공지사항으로 이동한다.

일반 사용자 플로우는 다음과 같다.



티켓팅 사용자 플로우

티켓팅 기능은 학생 인증 및 로그인을 완료한 사용자만 이용할 수 있도록 설계하였다. 사용자는 로그인 후 이벤트를 선택하고 대기열을 거쳐 예매 가능 상태가 되면 티켓을 예매한다.

티켓팅 사용자 플로우는 다음과 같다.



관리자 플로우

관리자는 별도의 관리자 페이지에 로그인한 뒤 운영 기능에 접근한다. 관리자 페이지에서는 공지사항 관리, 광고 배너 관리, 지도 관리, 이벤트 관리, 티켓 발급 처리를 수행할 수 있다.

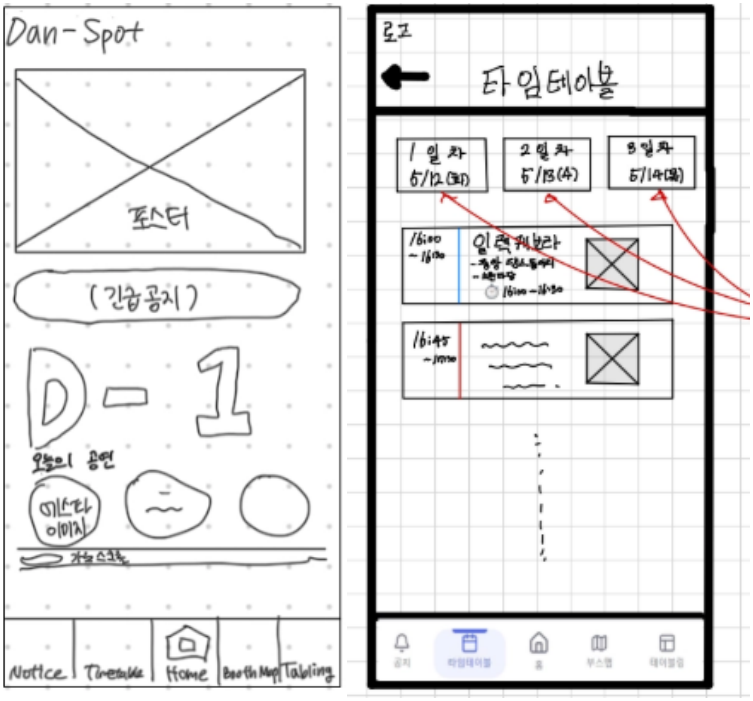
관리자 플로우는 다음과 같다.



4. 화면 설계

화면 설계는 모바일 사용 환경을 중심으로 구성하였다. 대학 축제 서비스는 사용자가 축제 현장에서 스마트폰으로 빠르게 정보를 확인하는 상황이 많기 때문에 주요 기능을 하단 내비게이션과 단순한 화면 구조를 통해 접근할 수 있도록 설계하였다.

- **홈 및 타임테이블**

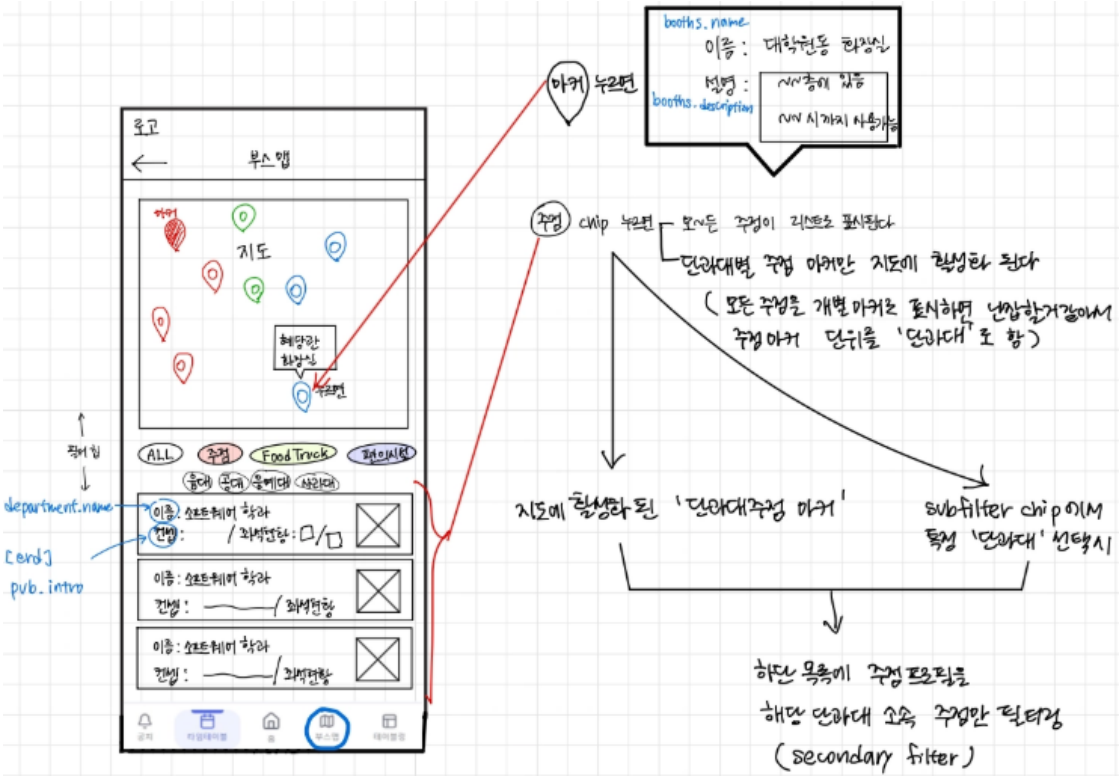


위 와이어프레임은 홈 화면과 타임테이블 화면의 기본 구성과 정보 배치 방식을 시각화한 설계안이다.



위 화면은 해당 와이어프레임을 바탕으로 구현한 실제 홈 화면 및 타임테이블 화면이다.

- **부스맵**

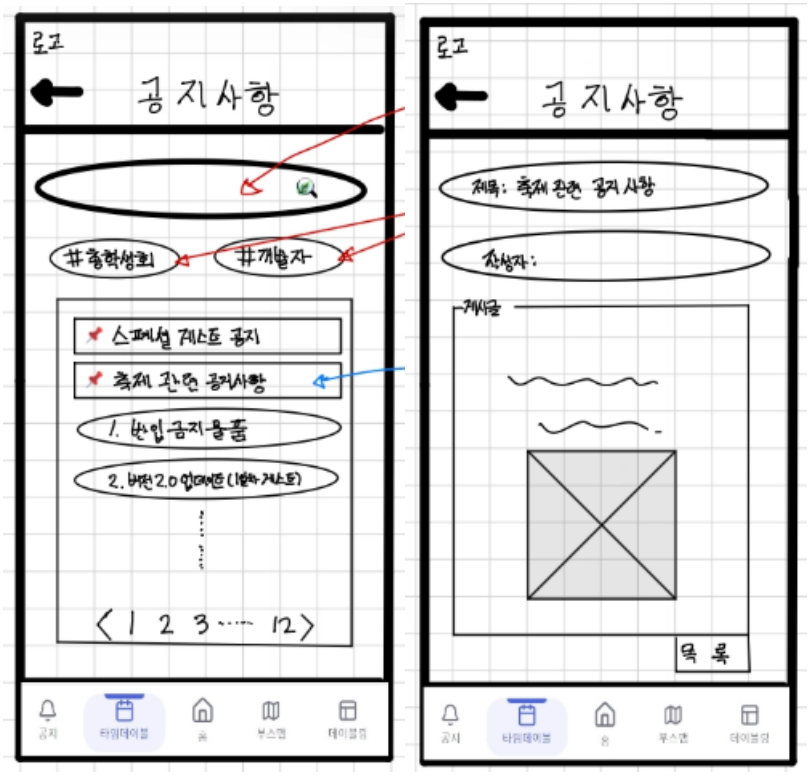


위 와이어프레임은 부스맵 화면의 정보 배치와 탐색 구조를 시각화한 설계안이다.

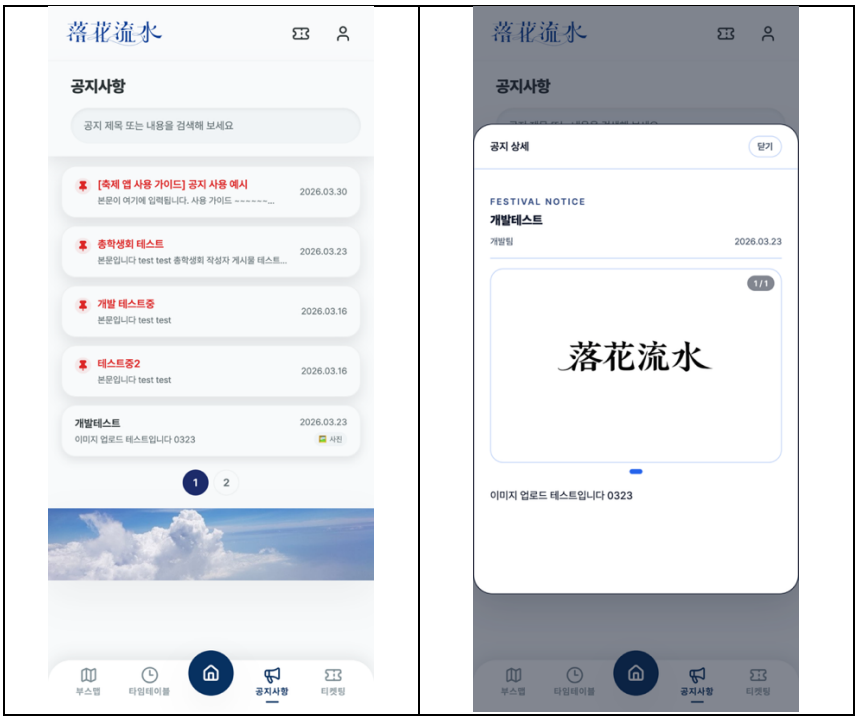


위 화면은 부스맵 기능의 실제 구현 화면으로, 지도 기반 탐색 구조를 띈다.

- **공지사항**



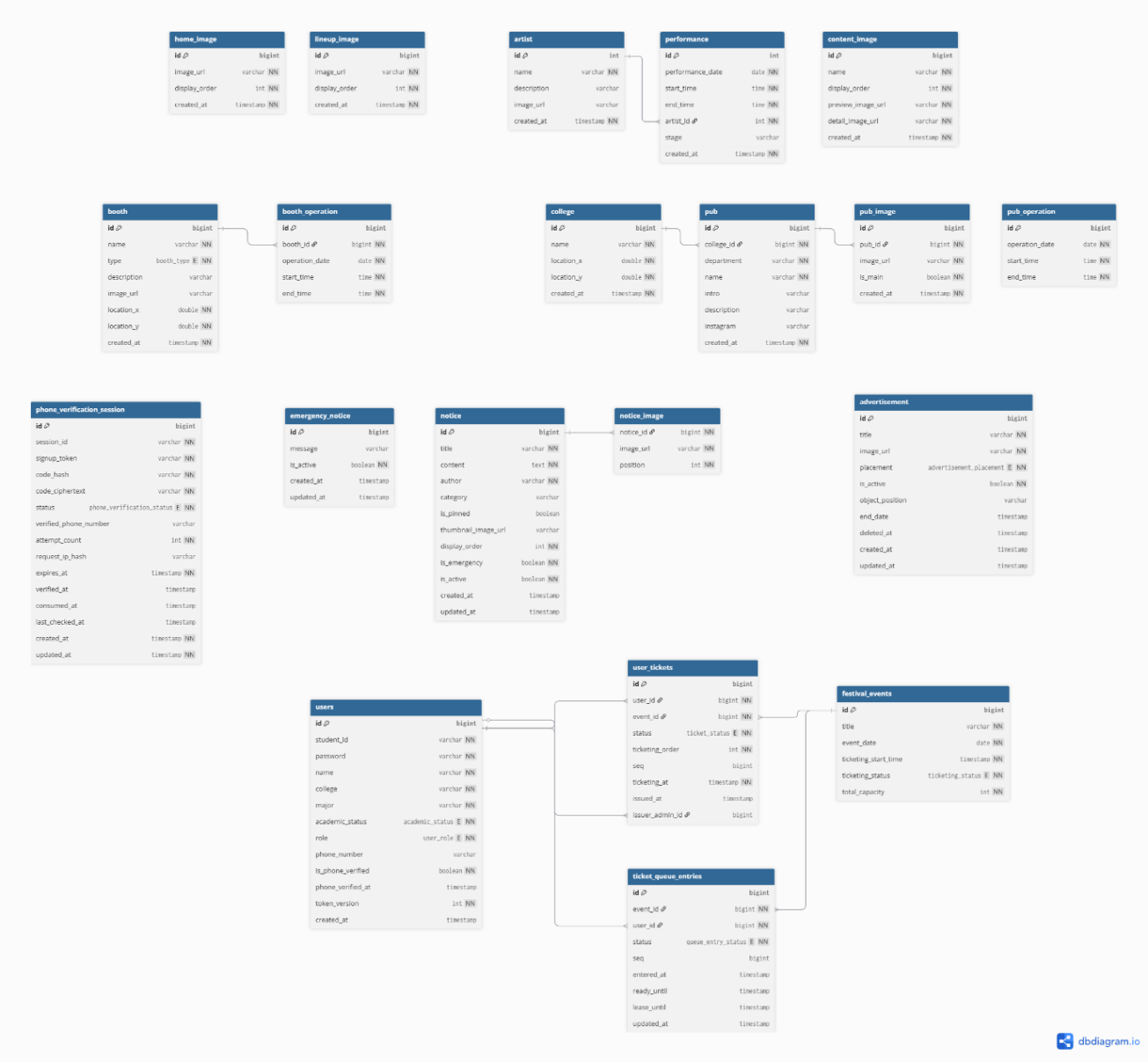
위 와이어프레임은 공지사항 화면의 정보 전달 구조와 화면 흐름을 정리한 설계안이다.



위 화면은 공지사항 목록 및 상세 조회 기능을 구현한 실제 서비스 화면이다.

5. 데이터베이스 설계

데이터베이스는 사용자 및 인증, 축제 콘텐츠, 부스맵, 공지 및 광고, 티켓팅 도메인으로 구분하여 설계하였다. 각 도메인은 서비스 기능과 데이터의 역할을 기준으로 분리하였으며 기능 확장과 유지보수가 용이하도록 구성하였다.



사용자 및 인증 도메인은 회원 정보와 인증 정보를 관리한다. 사용자 계정, 학생 정보, 전화번호 인증 상태, 권한 정보 등을 저장하여 회원가입, 로그인, 사용자 인증, 관리자 권한 구분에 활용하였다.

축제 콘텐츠 도메인은 홈 이미지, 라인업 이미지, 아티스트, 공연 정보를 관리한다. 이를 통해 사용자가 홈 화면과 타임테이블에서 축제의 주요 이미지와 공연 일정을 확인할 수 있도록 한다.

부스맵 도메인은 부스, 주점, 단과대, 운영 시간, 위치 정보를 관리한다. 부스와 주점의 좌표를 저장하여 지도 화면에서 위치를 표시하고 운영 날짜와 시간 정보를 함께 제공할 수 있도록 설계하였다.

공지 및 광고 도메인은 운영진이 등록하는 안내 콘텐츠를 관리한다. 일반 공지, 긴급 공지, 공지 이미지, 광고 배너 정보를 저장하여 사용자 화면에 필요한 안내와 홍보 콘텐츠를 노출할 수 있도록 한다.

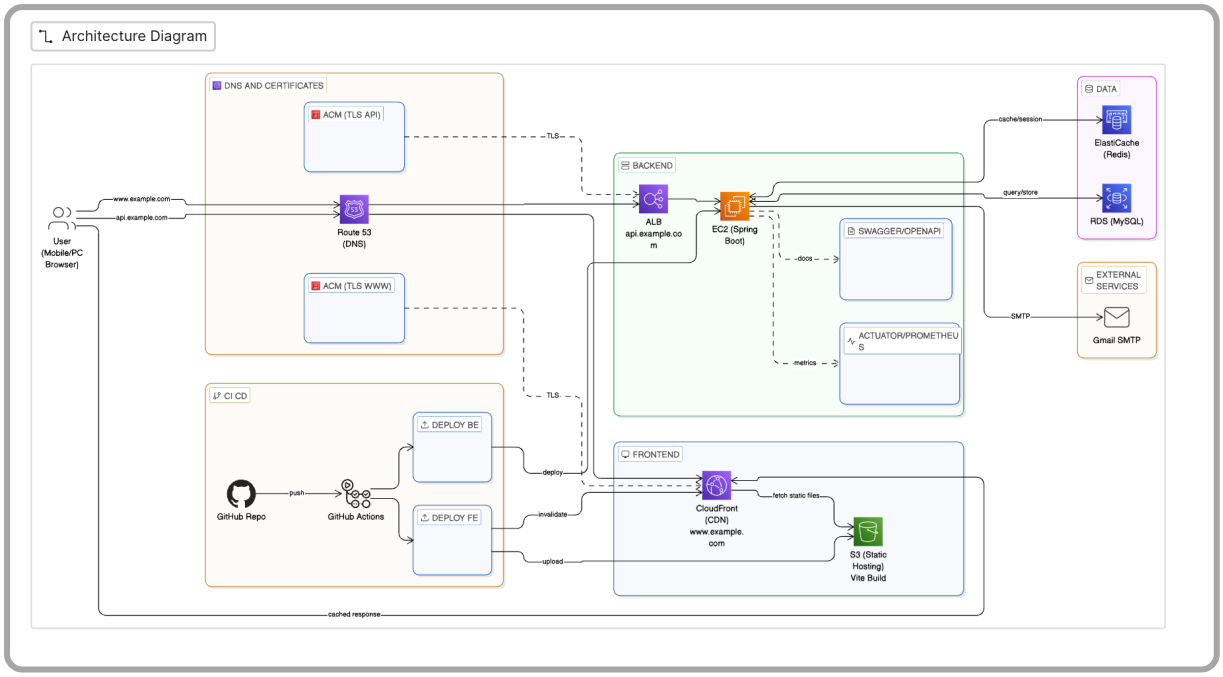
티켓팅 도메인은 티켓팅 이벤트, 대기열, 사용자 티켓 정보를 관리한다. 이벤트 정보와 총 수량을 저장하고 사용자의 대기열 상태와 예매 완료 티켓을 관리하여 예매와 현장 발급 흐름이 연결되도록 설계하였다.

핵심 테이블 설계는 다음과 같이 정리하였다.

테이블명	주요 필드	역할	제약 및 특징
users	user_id, student_id, role, phone	회원 계정과 권한 정보를 관리	사용자/관리자 권한 구분 및 인증 흐름과 연계
festival_events	event_id, title, event_date, ticketing_start_time, total_capacity	티켓팅 대상 행사 정보를 관리	행사별 일정과 총 수량 기준 저장
user_tickets	ticket_id, user_id, event_id, status, ticketing_order, issued_at	사용자 예매 결과와 현장 발급 상태를 관리	(user_id, event_id) 유일성 제약으로 중복 예매 방지
ticket_issue_requests	request_id, event_id, user_id, status, error_code	비동기 발급 요청 상태를 추적	요청 단위 이력 관리 및 보상 처리 연계
outbox_events	aggregate_id, topic, event_key, payload, status	메시지 발행 전 이벤트 저장	메시지 발행과 DB 저장 간 정합성 확보

6. 시스템 아키텍처 설계

DANZZAN 의 시스템은 프론트엔드, 백엔드 API 서버, 데이터베이스, Redis, 파일 저장소, 외부 서비스로 구성된다. 프론트엔드는 사용자와 관리자가 접근하는 웹 화면을 제공하며 백엔드는 인증, 축제 정보 조회, 티켓팅, 관리자 기능을 처리한다.



프론트엔드는 사용자 화면과 관리자 화면을 제공한다. 사용자는 웹 화면을 통해 축제 정보를 확인하고 티켓팅에 참여하며 관리자는 관리자 페이지를 통해 공지, 광고, 지도, 티켓 발급 상태를 관리한다.

백엔드 API 서버는 프론트엔드의 요청을 받아 필요한 데이터를 조회하거나 변경한다. 사용자 인증, 공지사항 조회, 공연 일정 조회, 부스맵 조회, 티켓팅 이벤트 처리, 관리자 기능 처리를 담당하며, 데이터베이스와 Redis 를 연동하여 서비스 흐름을 제어한다.

MySQL 은 서비스 운영에 필요한 영속 데이터를 저장한다. 사용자 정보, 공지사항, 공연 정보, 부스 및 주점 정보, 광고 정보, 티켓팅 이벤트, 사용자 티켓 정보가 MySQL 에 저장된다. Redis 는 티켓팅 대기열과 상태 관리에 사용된다. 티켓팅 기능은 특정 시간에 사용자가 몰리는 특성이 있기 때문에 Redis 를 활용하여 대기 순번과 예매 가능 상태를 빠르게 관리할 수 있도록 하였다.

이미지와 광고 배너 파일은 별도의 파일 저장소에 저장하여 서비스 화면에서 조회할 수 있도록 구성한다. 이를 통해 홈 이미지, 라인업 이미지, 공지 이미지, 광고 이미지 등 다양한 정적 파일을 효율적으로 관리할 수 있다.

전체적으로 DANZZAN 의 시스템 아키텍처는 사용자 서비스와 관리자 서비스를 하나의 백엔드 API 서버와 데이터 저장 구조로 연결하는 방식으로 설계하였다. 이를 통해 축제 정보 제공, 티켓팅, 현장 발급 처리, 관리자 운영 관리가 하나의 서비스 흐름 안에서 동작할 수 있도록 하였다.

7. 서버 구성 및 배포 구조 설계

DANZZAN의 서버 구성은 프론트엔드, 백엔드 API 서버, MySQL, Redis, Kafka, S3, 모니터링 서버로 구분하여 설계하였다. 프론트엔드는 React와 TypeScript 기반의 웹 애플리케이션으로서 사용자 화면과 관리자 화면을 제공하며, 백엔드 API 서버는 Spring Boot 3를 기반으로 인증, 공지, 지도, 티켓팅, 관리자 기능을 처리하도록 구성하였다. MySQL은 사용자 정보, 축제 콘텐츠, 티켓 정보, 관리자 처리 이력과 같은 영속 데이터를 저장하고, Redis는 대기열 순번, 예매 가능 상태, 만료 시간과 같이 짧은 주기로 변하는 상태 정보를 신속하게 관리하는 용도로 활용하였다.

티켓 발급 처리에는 Kafka를 활용한 비동기 구조를 적용하였다. 사용자가 예매 가능 상태에서 발급 요청을 전송하면 서버는 해당 요청을 별도의 발급 요청 단위로 저장한 후 메시지 브로커를 통해 후속 발급 절차가 수행되도록 설계하였다. 이러한 구조는 티켓팅 시점에 모든 비즈니스 로직이 동기적으로 집중되는 현상을 완화하고, 요청 수가 급격히 증가하는 상황에서도 발급 작업을 보다 안정적으로 분산 처리할 수 있도록 한다.

정적 파일은 S3에 저장하여 홈 이미지, 라인업 이미지, 공지 이미지, 광고 배너와 같은 파일 자원을 애플리케이션 서버와 분리하여 관리하도록 하였다. 또한 Prometheus와 Grafana를 통해 애플리케이션 및 Redis의 상태를 수집하고 시각화함으로써 티켓팅 시점에 발생할 수 있는 성능 저하나 병목 구간을 신속하게 파악할 수 있는 운영 구조를 마련하였다.

8. 데이터 형식 및 API 계약 설계

프론트엔드와 백엔드 간 연동의 안정성을 확보하기 위해 주요 API는 요청 및 응답의 데이터 형식을 명확히 구분하여 설계하였다. 일반적인 성공 응답은 success, data, error 구조를 기준으로 구성하였으며, 실패 응답은 message, status, errors 필드를 통해 오류 원인과 검증 실패 항목을 전달하도록 정의하였다. 이러한 공통 응답 규칙은 화면 개발 과정에서 예외 처리 로직을 일관된 방식으로 유지하는 데 기여한다.

티켓팅 API는 일반 조회형 API보다 상세한 상태 정보를 제공하도록 설계하였다. 예를 들어 대기열 상태 조회 응답에는 현재 상태, 대기열 순번, 선행 대기 인원 수, 예상 대기 시간, READY 상태 만료 시각 등의 값을 포함시켜 사용자가 자신의 현재 진행 단계를 명확히 인지할 수 있도록 하였다. 또한 상태 값은 WAITING, READY, ACTIVE, DONE과 같은 명시적 열거형으로 정의하여 프론트엔드와 백엔드가 동일한 의미 체계를 공유하도록 구성하였다.

날짜 및 시간 데이터는 정보의 성격에 따라 형식을 구분하여 설계하였다. 축제 일정과 같은 도메인 정보

에는 LocalDate 또는 LocalDateTime 기반 값을 사용하였고, 대기열 만료 시각이나 요청 접수 시각과 같이 프론트엔드에서 즉시 계산이 필요한 값은 epoch milliseconds 형식으로 전달하도록 하였다. 이를 통해 일정 정보와 실시간 제어 정보를 명확히 구분하고, 사용자 화면에서 남은 시간 계산이나 폴링 제어를 단순화할 수 있도록 하였다.

9. DTO 설계

DTO 설계는 엔티티와 외부 API 계약을 분리하기 위한 목적으로 구성하였다. 엔티티는 데이터베이스 저장 구조를 표현하고, DTO는 프론트엔드와의 요청 및 응답 형식을 표현함으로써 내부 모델이 외부 인터페이스에 직접 노출되지 않도록 하였다. 이를 통해 계층 간 결합도를 낮추고 필드 확장이나 응답 형식 변경에 유연하게 대응할 수 있도록 하였다.

대표 DTO 구성은 다음과 같다.

DTO명	구분	주요 필드	설계 목적
RequestLoginDto	요청 DTO	studentId, password	로그인 요청 시 필수 입력값 전달
ResponseLoginDto	응답 DTO	accessToken, refreshToken, user	로그인 성공 후 토큰과 사용자 요약 정보 반환
TicketStatusResponseDTO	응답 DTO	status, queuePosition, aheadCount, readyUntil	대기열 상태 조회 결과를 세분화하여 전달
TicketRequestResponseDTO	응답 DTO	status, requestId, acceptedAt, remaining	대기열 진입 및 예매 요청 처리 결과 반환
ResponseMyTicketListDto	응답 DTO	tickets[]	사용자의 예매 내역 목록 전달
TicketIssueRequestStatusResponseDTO	응답 DTO	requestId, status, errorCode, updatedAt	비동기 발급 요청 상태 추적 결과 반환

10. 주요 API 명세

핵심 API는 사용자 인증, 대기열 제어, 티켓 발급, 사용자 정보 조회 흐름을 기준으로 선정하였다. 각 API는 요청 목적과 응답 DTO를 명확히 구분하여 프론트엔드와 백엔드 간 연동 계약을 일관되게 유지하도록 설계하였다.

주요 API 명세는 다음과 같다.

메서드	엔드포인트	기능	주요 응답 DTO
POST	/user/login	학번/비밀번호 기반 로그인	ResponseLoginDto
GET	/tickets/events	티켓팅 가능 이벤트 목록 조회	ResponseTicketEventListDto
POST	/tickets/{eventId}/queue/enter	대기열 진입	TicketRequestResponseDTO
GET	/tickets/{eventId}/queue/status	대기열 상태 조회	TicketStatusResponseDTO
POST	/tickets/{eventId}/activate	READY 상태에서 ACTIVE 상태 전환	TicketRequestResponseDTO
POST	/tickets/{eventId}/reserve	티켓 예매 요청	ResponseReserveTicketDto 또는 TicketRequestResponseDTO
GET	/user/me	현재 로그인 사용자 정보 조회	ResponseLoginDto.UserInfo

11. 예외 응답 설계

예외 응답 설계에서는 인증 실패, 권한 부족, 재고 소진, 중복 예매, 입력값 검증 실패와 같은 주요 실패 시나리오에 대해 상태 코드와 응답 메시지를 명시적으로 정의하였다. 이를 통해 프론트엔드는 실패 유형을 구분하여 적절한 안내 문구를 제공할 수 있고, 운영 측면에서는 오류 원인을 보다 일관되게 추적할 수 있도록 하였다.

주요 예외 응답은 다음과 같다.

구분	HTTP 상태	응답 코드/필드	설명
로그인 실패	401	error	학번 또는 비밀번호 불일치 시 반환
권한 없음	403	FORBIDDEN	관리자 권한이 없는 요청 차단
이벤트 미오픈	400	RESERVE_NOT_OPEN	READY 또는 ACTIVE 조건 미충족 시 반환
재고 소진	409	RESERVE_SOLD_OUT	잔여 티켓이 없을 때 반환
중복 예매	409	RESERVE_ALREADY_RESERVED	동일 사용자의 중복 예매 요청 차단
검증 실패	400	message, errors[]	필수값 누락 또는 형식

			오류 전달

구분	형식 예시	설계 의도
성공 응답	{ success: true, data: ... , error: null }	클라이언트가 정상 응답과 예외 응답을 일관되게 분기하도록 구성
실패 응답	{ message, status, errors[] }	오류 원인과 검증 실패 필드를 명시적으로 전달
티켓 대기 상태	WAITING / READY / ACTIVE / DONE	대기열 상태 전이를 프론트엔드와 백엔드가 동일하게 해석하도록 정의
실시간 시각 값	epoch milliseconds	남은 시간 계산 및 폴링 제어를 단순화
일정 정보	LocalDate / LocalDateTime	축제 일정과 실시간 제어 정보를 구분하여 표현

9. 티켓팅 정합성 설계

티켓팅 기능은 단순한 Redis 대기열 처리에 그치지 않고, Redis와 MySQL, Kafka를 결합한 정합성 중심 구조로 설계하였다. 사용자가 대기열을 통과한 뒤 예매를 시도하면 서버는 현재 상태, 중복 예매 여부, 잔여 수량을 우선 확인하고 발급 요청을 고유한 requestId 단위로 기록한다. 이후 발급 요청 정보는 outbox 형태로 함께 저장하여 메시지 발행과 데이터 저장 사이의 불일치 가능성을 최소화하도록 하였다.

비동기 소비자는 발급 요청 메시지를 수신하여 실제 티켓 발급을 수행하고, 처리 성공 시 사용자 티켓을 생성한 뒤 대기열 상태를 DONE으로 변경하도록 하였다. 이 과정에서는 사용자와 이벤트 조합에 대한 유일성 제약을 적용하여 동일 사용자가 동일 이벤트를 중복 예매하지 못하도록 하였으며, 발급 실패 시에는 보상 처리 로직을 통해 요청 상태와 재고 상태가 불일치하지 않도록 설계하였다. 이러한 구조는 티켓팅 시점의 순간 부하를 흡수하는 동시에 최종 데이터 정합성을 유지하는 데 목적이 있다.

또한 대기열 상태 변경과 만료 처리에는 Lua 스크립트를 활용하여 Redis 내부에서 원자적으로 수행되도록 구성하였다. READY 상태의 사용자가 제한 시간 내에 진입하지 못할 경우 해당 상태를 만료시키고 다음 사용자를 입장시키는 흐름, ACTIVE 상태 사용자의 예매 세션 만료 처리, 상태 스냅샷 조회 등을 서버 코드와 분리된 스크립트로 관리함으로써 동시성 상황에서도 상태 전이가 일관되게 유지되도록 하였다.

10.Docker Compose 기반 인프라 설계

개발 및 통합 테스트 환경에서는 Docker Compose를 활용하여 애플리케이션과 지원 인프라를 일관된 방식으로 실행하도록 구성하였다. 애플리케이션 스택에는 Spring Boot 애플리케이션 컨테이너, MySQL,

Redis, Kafka, Kafka 초기화 컨테이너가 포함되며, 각 컨테이너는 헬스체크와 depends_on 조건을 통해 기동 순서를 조정하도록 하였다. 이를 통해 팀원은 로컬 환경에서 동일한 구성을 신속하게 재현할 수 있으며, 티켓팅 기능과 같이 의존 서비스가 많은 구조도 안정적으로 실행할 수 있다.

운영 보조 및 성능 검증 환경은 별도의 Compose 파일로 구성하였다. Redis Exporter, Prometheus, Grafana를 분리하여 메트릭 수집과 시각화 기능을 독립적으로 운영하고, 필요 시 DB SSH 터널 컨테이너를 통해 외부 데이터베이스 상태를 점검할 수 있도록 하였다. 이와 같이 애플리케이션 실행 환경과 모니터링 환경을 분리함으로써 실제 서비스 흐름과 관찰 도구가 상호 간섭을 최소화하도록 설계하였다.

백엔드 애플리케이션은 Dockerfile을 통해 Java 17 기반의 멀티 스테이지 빌드 구조로 컨테이너화하였다. 빌드 단계에서는 Gradle을 이용하여 bootJar를 생성하고, 실행 단계에서는 JRE 이미지만 사용하여 런타임 이미지를 단순화하였다. 이러한 방식은 배포 산출물의 일관성을 높이고, 팀원별 개발 환경 차이로 인한 실행 문제를 줄이는 데 유리하다.

Docker Compose 기반 주요 서비스 구성은 다음과 같이 정리하였다.

서비스	역할	주요 포트	비고
app	Spring Boot API 서버 실행	8080	MySQL, Redis, Kafka 의존
mysql	영속 데이터 저장	3306	행사, 사용자, 티켓 정보 저장
redis	대기열 및 상태 캐시 관리	6379	Sorted Set 및 상태 키 저장
kafka	비동기 티켓 발급 메시지 브로커	9094	발급 요청 이벤트 전달
prometheus	애플리케이션 및 Redis 메트릭 수집	9090	/actuator/prometheus 스크랩
grafana	대시보드 시각화	3000	부하 테스트 및 운영 모니터링
redis-exporter	Redis 메트릭 노출	9121	Prometheus 연동용

11.인증 및 보안 설계

인증 구조는 사용자 영역과 관리자 영역을 구분하여 설계하였다. 일반 사용자는 JWT 기반 로그인 세션을 통해 서비스 기능에 접근하며, 관리자 기능은 별도의 관리자 인증 절차를 거쳐 공지 관리, 지도 관리, 광고 관리, 티켓 발급 처리 기능에 접근하도록 구성하였다. 이를 통해 동일한 서비스 내부에서도 권한 경계

를 명확히 분리하고, 민감한 운영 기능이 일반 사용자에게 노출되지 않도록 하였다.

또한 회원가입 과정에는 학생 정보 확인, 전화번호 인증, 비밀번호 정책 검증과 같은 절차를 포함하여 계정의 신뢰성을 높이고자 하였다. 비밀번호 재설정 기능 역시 별도의 요청 ID와 인증 코드 검증 흐름을 두어 일회성 검증이 완료된 경우에만 비밀번호 변경이 가능하도록 설계하였다. 현장 발급 기능에서는 어떤 관리자가 언제 티켓을 발급하였는지를 기록하여 운영 이력을 추적할 수 있도록 하였다.

클라이언트와 서버 간 통신에서는 CORS, Authorization 헤더 처리, 토큰 만료 검증을 고려하여 보안 구성을 적용하였다. 특히 축제 기간에는 외부 네트워크 환경과 모바일 접속 비중이 높기 때문에 단순히 로그인 기능을 구현하는 수준에 그치지 않고, 인증 실패나 만료 상황에서도 서비스가 예측 가능한 방식으로 동작하도록 예외 응답 구조를 함께 설계하였다.

IV. 구현 기술 및 개발 환경

1. 주요 기술 스택

DANZZAN 은 사용자 화면, 관리자 화면, 백엔드 API, 데이터 저장소, 티켓팅 대기열을 안정적으로 구성하기 위해 기능별로 적합한 기술을 선정하였다. 프론트엔드는 React 와 TypeScript 를 기반으로 구현하였고, 백엔드는 Spring Boot 3 와 Java 17 을 사용하여 REST API 서버를 구축하였다. 데이터 저장소는 MySQL 을 중심으로 구성하였으며, 티켓팅과 같이 빠른 상태 변경과 대기열 처리가 필요한 기능에는 Redis 를 활용하였다.

또한 사용자와 관리자의 인증 상태를 관리하기 위해 JWT 기반 인증 방식을 적용하였고, 축제 이미지, 공지 이미지, 광고 배너와 같은 정적 파일은 S3 를 활용하여 관리하도록 구성하였다. 프론트엔드에서는 서버 상태 관리와 API 캐싱을 위해 TanStack Query 를 사용하였으며, 로그인 정보나 티켓팅 진행 상태처럼 화면 간 공유가 필요한 값은 Zustand 를 통해 관리하도록 하였다.

기술	적용 영역	선정 이유
React + TypeScript	프론트엔드	컴포넌트 기반 UI 구성과 타입 안정성 확보
Spring Boot 3	백엔드	REST API 서버 구현 및 인증·인가 처리에 적합
Java 17	백엔드 개발 언어	Spring Boot 3 환경과 안정적으로 호환
MySQL	데이터베이스	사용자, 공지, 티켓 등 영속 데이터 관리
Redis	대기열 및 상태 관리	빠른 읽기·쓰기와 티켓팅 대기 순번 관리에 적합
JWT	인증 및 인가	사용자와 관리자 권한 구분 및 로그인 상태 관리
S3	파일 저장	이미지, 광고 배너 등 정적 파일 관리
TanStack Query	서버 상태 관리	API 데이터 캐싱 및 대기열 상태 폴링 처리
Zustand	클라이언트 상태 관리	인증 세션, 티켓팅 흐름 등 전역 상태 관리

이와 같은 기술 조합을 통해 사용자에게는 빠르고 안정적인 화면 경험을 제공하고, 운영진에게는 공지, 광고, 지도, 티켓 발급 상태를 효율적으로 관리할 수 있는 환경을 마련하였다. 특히 Redis, TanStack Query, Zustand 는 티켓팅과 같이 실시간 상태 변화가 중요한 기능에서 사용자 상태를 안정적으로 관리하기 위한 목적으로 활용하였다.

2. 티켓팅 대기열 구현 방식

DANZZAN 의 티켓팅 기능은 특정 시간에 많은 사용자가 동시에 접근할 수 있는 상황을 고려하여 Redis 기반 대기열 구조로 설계하였다. 사용자가 티켓팅 이벤트에 접근하면 즉시 예매 화면으로 이동하는 것이 아니라 먼저 대기열에 진입하도록 하였고, 이후 서버는 대기 순서에 따라 일부 사용자만 예매 가능 상태로 전환하여 해당 사용자만 실제 예매를 진행할 수 있도록 구성하였다.

대기열 순번 관리는 Redis 의 Sorted Set 을 활용하였다. Sorted Set 은 사용자를 점수 기준으로 정렬할 수 있으므로 대기열 진입 순서나 순번을 관리하는 데 적합하다. 사용자의 현재 상태는 Redis Hash 를 활용하여 관리하도록 하였으며, 상태는 WAITING, READY, ACTIVE, DONE 단계로 구분하여 각 단계가 대기 중, 예매 가능 대기, 예매 진행, 예매 완료 또는 종료 상태를 의미하도록 정의하였다.

상태	의미
WAITING	사용자가 대기열에 진입했지만 아직 예매 가능 상태가 아닌 상태
READY	예매 가능 대상자로 선정되었으나 예매 화면 진입 전 상태
ACTIVE	예매 화면에 진입하여 실제 예매를 진행할 수 있는 상태
DONE	예매가 완료되었거나 대기열 처리가 종료된 상태

티켓팅 화면에서는 사용자의 현재 대기 상태를 주기적으로 확인해야 하므로 프론트엔드에서는 TanStack Query 를 활용하여 대기열 상태를 폴링하도록 구성하였다. 이를 통해 사용자는 자신의 대기 순번, 예매 가능 여부, 예매 화면 진입 가능 상태를 화면에서 확인할 수 있도록 하였다.

예매 요청이 발생하면 서버는 사용자의 대기열 상태, 중복 예매 여부, 티켓 잔여 수량을 확인한 뒤 예매를 처리하도록 하였다. 예매가 성공하면 사용자 티켓 정보가 데이터베이스에 저장되고 해당 사용자의 대기열 상태는 DONE 으로 변경되도록 구성하였다. 재고 차감과 예매 상태 변경과 같이 정합성이 중요한 작업은 하나의 흐름으로 처리하여 동시 요청 상황에서도 중복 예매나 수량 불일치가 발생하지 않도록 설계하였다.

대기열 진입 → 대기 상태 확인 → 예매 가능 상태 전환 → 티켓 예매 → 내 티켓 확인

이러한 대기열 구조를 통해 티켓팅 시작 시 서버에 요청이 한 번에 집중되는 상황을 완화하고, 예매 순서와 티켓 수량을 보다 안정적으로 관리할 수 있도록 하였다. 또한 예매 완료 후에는 내 티켓 화면과 관리자 티켓 발급 처리 화면으로 데이터가 연결되어 사용자 예매 흐름과 현장 운영 흐름이 이어지도록 설계하였다.

3. 개발 및 협업 환경

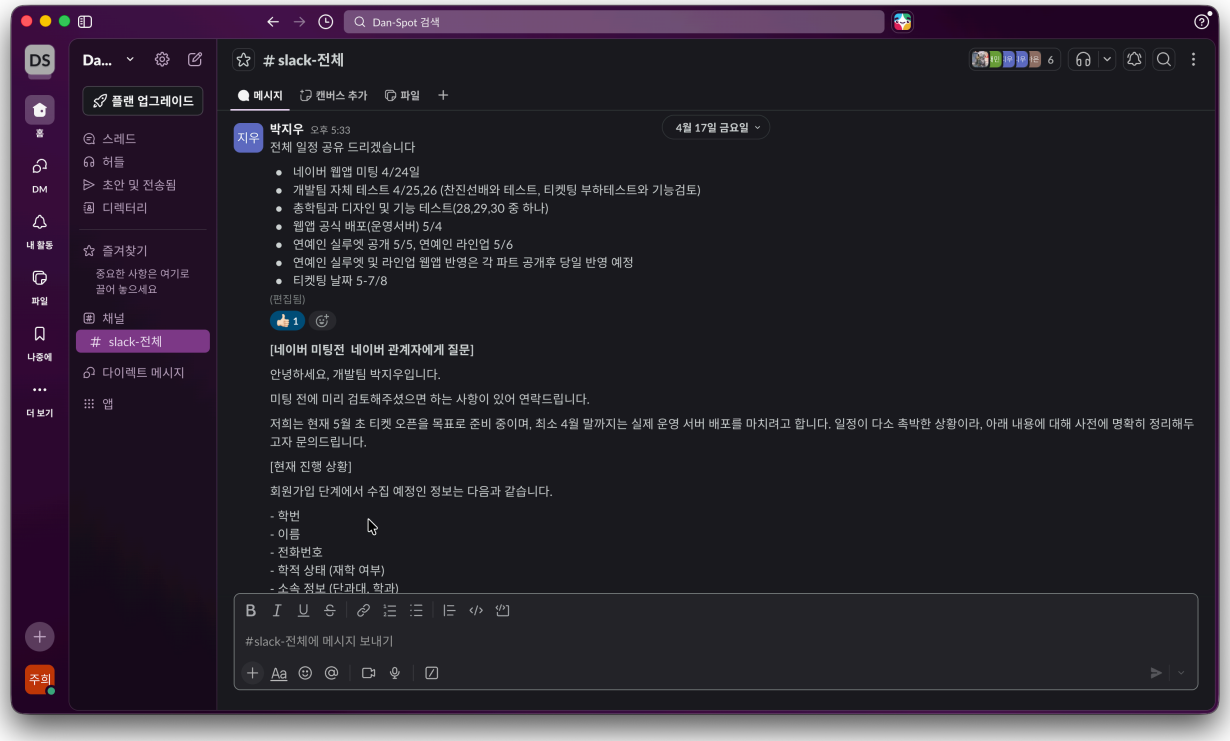
DANZZAN 은 팀 단위 개발을 효율적으로 진행하기 위해 개발 도구와 협업 도구를 구분하여 사용하였다. 코드 관리는 GitHub 를 활용하여 기능별 브랜치를 생성한 뒤 개발을 진행하도록 하였으며, 개발이 완료된 기능은 Pull Request 를 통해 코드 리뷰를 거친 후 develop 브랜치에 병합하도록 운영하였다. 이후 배포가 필요한 경우에는 main 브랜치에 반영하여 운영 환경에 적용하는 흐름을 따르도록 하였다.

API 명세와 테스트에는 Swagger UI 를 활용하였다. 이를 통해 백엔드에서 구현한 API 를 문서 형태로 확인하고, 프론트엔드와 백엔드 간 요청 및 응답 형식을 조율할 수 있도록 하였다. 일정 관리와 기획 문서 정리에는 Notion 과 Jira 를 사용하여 회의록, 작업 항목, 스프린트 진행 상황을 기록하였으며, 팀원 간 실시간 소통과 개발 알림 공유에는 Slack 과 Discord 를 활용하였다.

1. 커뮤니케이션 체계

팀 내 실시간 의사소통은 Slack 을 중심으로 운영하였다. 기능 개발 중 발생한 이슈, 일정 변경, 배포 및 테스트 결과, 긴급 공지 사항을 채널별로 구분하여 공유함으로써 필요한 정보를 신속하게 확인할 수 있도록 하였다. 장문 논의나 회의 이후 정리된 내용은 Notion 에 문서화하여 구두 전달 과정에서 정보가 누락되지 않도록 관리하였다. 이를 통해 즉시성이 요구되는 소통과 기록이 필요한 소통을 분리하여 협업 효율을 높이고자 하였다.

[Slack 협업 채널 화면]



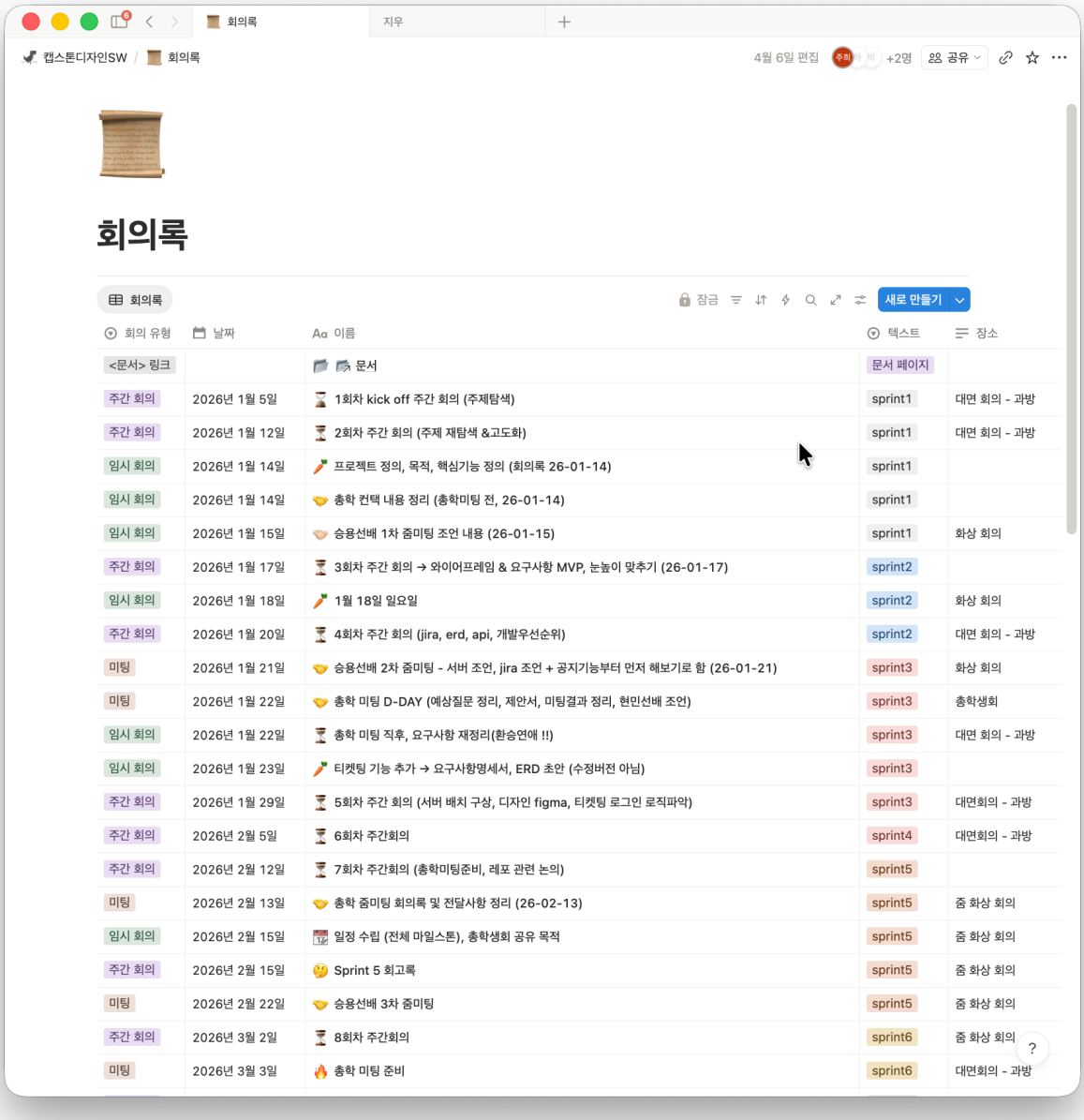
2. 일정 및 이슈 관리 방식

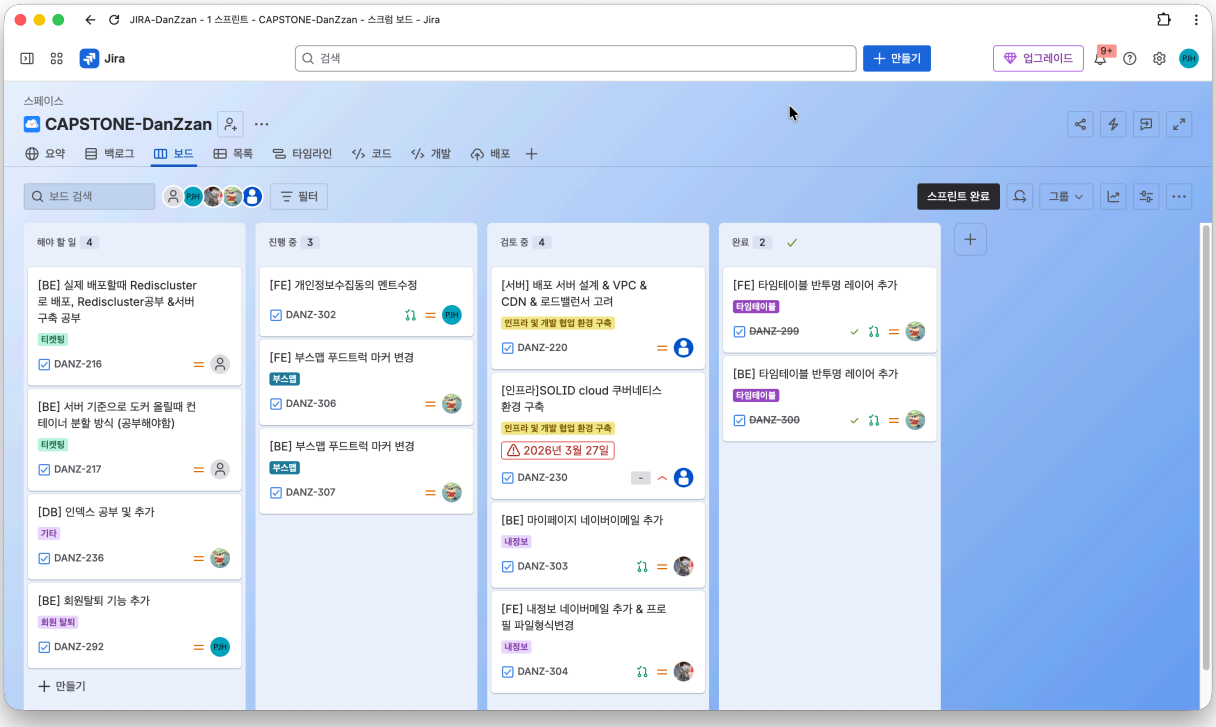
일정 및 작업 관리는 Notion 과 Jira 를 병행하였다. Notion 에는 요구사항, 화면 기획안, 회의록, 일정표, 참고 자료를 정리하여 프로젝트 전반의 문맥을 공유하였고, Jira 에는 기능별 작업 항목을 이슈 단위로 등록하여 우선순위와 진행 상태를 관리하였다. 각 이슈는 담당자, 목표 일정, 선행 작업 여부를 기준으로 정리하였으며, 스프린트 단위로 작업 범위를 조정하여 일정 지연이나 누락을 줄이고자 하였다.

특히 2026 년 1 월 20 일 프로젝트 초기에 Jira Epic 을 기반으로 전체 범위를 구상하였다. 당시에는 인프라 및 협업 환경 구축, 인터랙티브 축제 지도, 실시간 예약 및 대기 시스템, AI 축제 도우미, 운영자용

관리 대시보드, 브랜딩 및 디자인, 홈 화면 및 타임테이블 등으로 기능 범위를 나누어 상위 단위에서 작업을 분류하였다. 이후 실제 개발이 진행되면서 팀의 구현 우선순위와 축제 운영 적용 가능성을 반영하여 Epic 범위를 조정하였고, 최종적으로는 축제 정보 통합 제공, 티켓팅, 관리자 기능, 운영 안정성 확보에 보다 집중하는 방향으로 구체화하였다.

[Notion & Jira 협업 보드 화면]

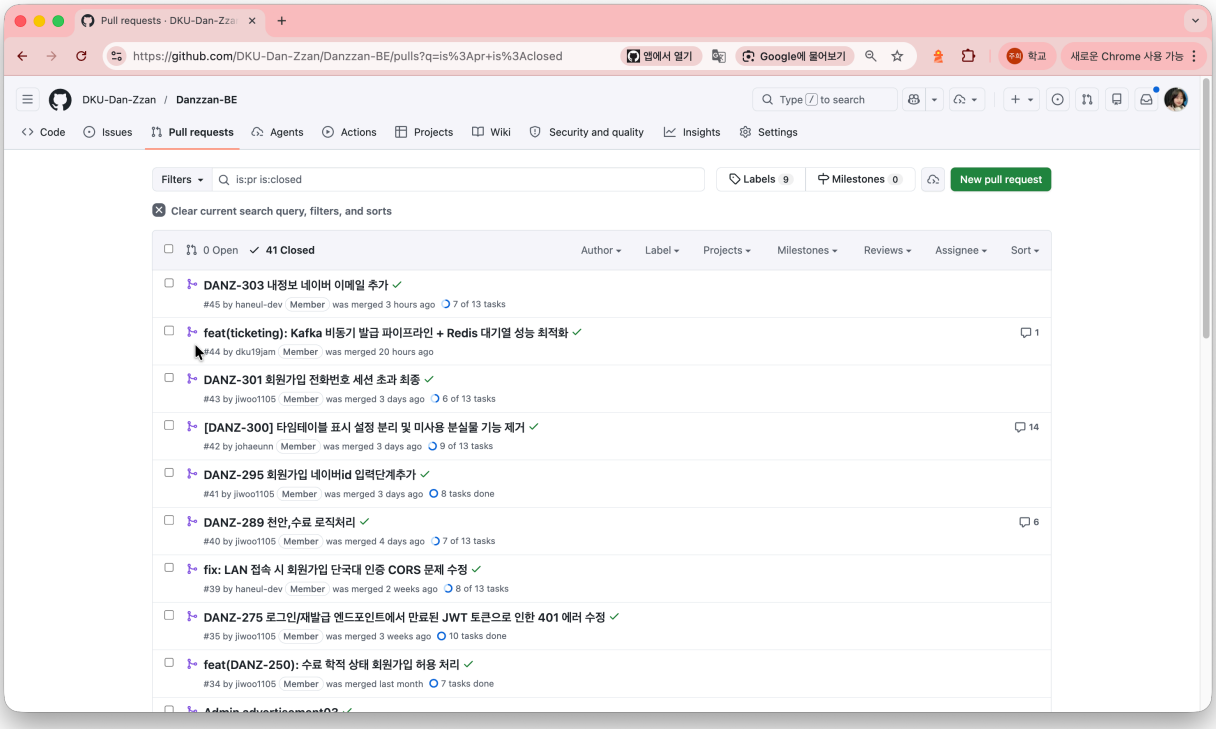


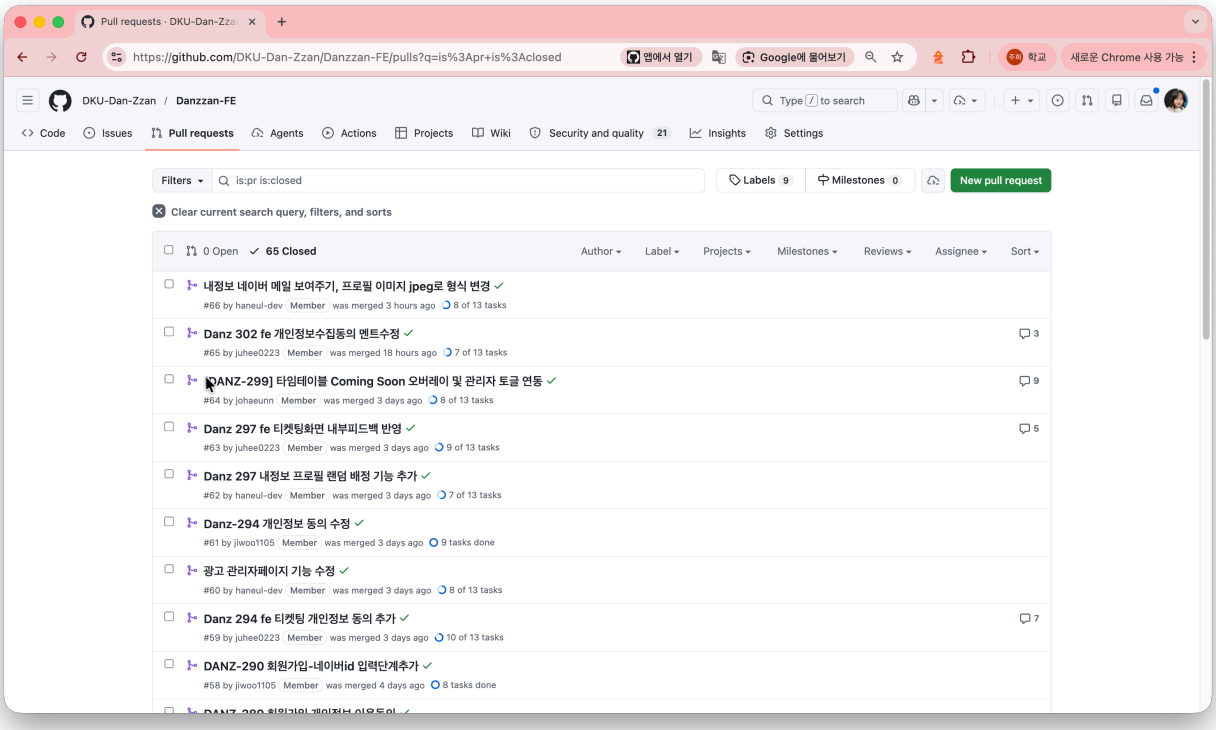


3. 브랜치 전략 및 코드 리뷰 프로세스

코드 협업은 GitHub 를 기반으로 기능 단위 브랜치를 생성하고 Pull Request 를 통해 변경 사항을 검토하는 방식으로 운영하였다. 브랜치 전략은 GitHub Flow 의 핵심 원칙을 참고하되 팀 개발에 맞게 develop 브랜치를 통합 브랜치로 두고, 기능 개발은 feature 브랜치에서 진행한 뒤 리뷰 완료 후 develop 에 병합하였다. 이후 테스트와 검증을 거친 결과물만 main 브랜치에 반영하여 배포 안정성을 확보하였다. 또한 Pull Request 에는 작업 목적, 주요 변경 사항, 테스트 결과를 함께 기록하여 리뷰어가 변경 의도를 빠르게 파악할 수 있도록 하였다.

[GitHub Pull Request 화면]


Pull requests · DKU-Dan-Zzan
https://github.com/DKU-Dan-Zzan/Danzzan-BE/pulls?q=is%3Apr+is%3Aclosed
DKU-Dan-Zzan / Danzzan-BE
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
Filters
is:pr is:closed
Labels 9
Milestones 0
New pull request
Clear current search query, filters, and sorts
0 Open
41 Closed
Author
Label
Projects
Milestones
Reviews
Assignee
Sort
DANZ-303 내정보 네이버 이메일 추가
#45 by haneul-dev Member was merged 3 hours ago 7 of 13 tasks
feat(ticketing): Kafka 비동기 발급 파이프라인 + Redis 대기열 성능 최적화
#44 by dku19jam Member was merged 20 hours ago
DANZ-301 회원가입 전화번호 세션 초과 최종
#43 by jiwoo1105 Member was merged 3 days ago 6 of 13 tasks
[DANZ-300] 타임테이블 표시 설정 분리 및 미사용 분실물 기능 제거
#42 by johaeunn Member was merged 3 days ago 9 of 13 tasks
DANZ-295 회원가입 네이버id 입력단계추가
#41 by jiwoo1105 Member was merged 3 days ago 8 tasks done
DANZ-289 천안,수료 로직처리
#40 by jiwoo1105 Member was merged 4 days ago 7 of 13 tasks
fix: LAN 접속 시 회원가입 단국대 인증 CORS 문제 수정
#39 by haneul-dev Member was merged 2 weeks ago 8 of 13 tasks
DANZ-275 로그인/재발급 엔드포인트에서 만료된 JWT 토큰으로 인한 401 에러 수정
#35 by jiwoo1105 Member was merged 3 weeks ago 10 tasks done
feat(DANZ-250): 수료 학적 상태 회원가입 허용 처리
#34 by jiwoo1105 Member was merged last month 7 tasks done


Pull requests · DKU-Dan-Zzan
https://github.com/DKU-Dan-Zzan/Danzzan-FE/pulls?q=is%3Apr+is%3Aclosed
DKU-Dan-Zzan / Danzzan-FE
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality 21
Insights
Settings
Filters
is:pr is:closed
Labels 9
Milestones 0
New pull request
Clear current search query, filters, and sorts
0 Open
65 Closed
Author
Label
Projects
Milestones
Reviews
Assignee
Sort
내정보 네이버 메일 보여주기, 프로필 이미지 jpeg로 형식 변경
#66 by haneul-dev Member was merged 3 hours ago 8 of 13 tasks
Danz 302 fe 개인정보수집동의 멘트수정
#65 by juhee0223 Member was merged 18 hours ago 7 of 13 tasks
[DANZ-299] 타임테이블 Coming Soon 오버레이 및 관리자 토글 연동
#64 by johaeunn Member was merged 3 days ago 8 of 13 tasks
Danz 297 fe 티켓팅화면 내부피드백 반영
#63 by juhee0223 Member was merged 3 days ago 9 of 13 tasks
Danz 297 내정보 프로필 랜덤 배정 기능 추가
#62 by haneul-dev Member was merged 3 days ago 7 of 13 tasks
Danz-294 개인정보 동의 수정
#61 by jiwoo1105 Member was merged 3 days ago 9 tasks done
광고 관리자페이지 기능 수정
#60 by haneul-dev Member was merged 3 days ago 8 of 13 tasks
Danz 294 fe 티켓팅 개인정보 동의 추가
#59 by juhee0223 Member was merged 3 days ago 10 of 13 tasks
DANZ-290 회원가입-네이버id 입력단계추가
#58 by jiwoo1105 Member was merged 4 days ago 8 tasks done

4. 데일리스크럼 운영

팀은 짧은 주기의 데일리스크럼 또는 정기 점검을 통해 진행 상황을 공유하였다. 각 팀원은 전일 완료한 작업, 당일 진행 예정 작업, 현재 막혀 있는 이슈를 중심으로 간단히 보고하였고, 논의가 길어질 사안은 별도 회의로 분리하여 전체 회의 시간을 최소화하였다. 이러한 방식은 작업 우선순위를 빠르게 재조정하고, 프론트엔드와 백엔드 간 의존 관계를 조기에 확인하며, 일정 리스크를 미리 대응하는 데 도움이 되었다.

5. 모니터링 및 부하 테스트 환경

티켓팅 기능은 짧은 시간에 다수의 사용자가 동시에 접근하는 구조이므로, 부하 테스트 과정에서 시스템 상태를 실시간으로 관찰할 수 있는 모니터링 환경을 함께 구성하였다. Prometheus 를 중심으로 메트릭을 수집하고 Grafana 를 통해 대시보드를 시각화함으로써 요청 증가 시점의 서버 상태, Redis 상태, 응답 지표 변화를 한 화면에서 확인할 수 있도록 하였다. 이를 통해 단순히 요청 성공 여부만 확인하는 것이 아니라 병목 구간과 자원 사용 추이를 함께 분석할 수 있도록 하였다.

monitoring/prometheus.yml 에서는 세 가지 대상을 수집하도록 구성하였다. 첫째, 백엔드 애플리케이션의 /actuator/prometheus 엔드포인트를 스크랩하여 애플리케이션 메트릭을 수집하였다. 둘째, Redis Exporter 를 통해 Redis 의 메모리 사용량, 연결 수, 명령 처리 상태와 같은 인프라 지표를 수집하였다. 셋째, Prometheus 자기 자신의 메트릭도 함께 수집하여 스크랩 상태와 모니터링 서버의 동작 여부를 점검할 수 있도록 하였다. Grafana 는 이렇게 수집된 데이터를 기반으로 티켓팅 부하 테스트 결과를 대시보드 형태로 시각화하는 역할을 수행하였다.

모니터링 환경의 주요 포트는 다음과 같이 구성하였다. 3306 포트는 데이터베이스 확인을 위한 DB SSH 터널에 사용하였고, 6379 포트는 Redis, 9121 포트는 Redis Exporter, 9090 포트는 Prometheus, 3000 포트는 Grafana 에 할당하였다. 이와 같은 구성을 통해 애플리케이션, 캐시, 모니터링 수집기, 시각화 도구를 분리된 포트에서 운영하며 부하 테스트 중 각 구성 요소의 상태를 독립적으로 점검할 수 있도록 하였다.

6. 스프린트 회고 및 협업 개선

팀은 스프린트 단위 회고를 통해 목표 달성도와 협업 방식을 점검하였다. 초기 스프린트에서는 목표 달성도를 약 95% 수준으로 평가하였고, 다음 스프린트에서는 계획 대비 추가 작업이 진행되어 120% 수준의 성과를 기록하였다. 회고 과정에서는 총학생회 및 외부 미팅을 안정적으로 수행한 점, Jira 와

회의록을 꾸준히 활용한 점, Pull Request 와 병합 과정을 실제로 경험하며 협업 흐름을 익힌 점을 주요 성과로 정리하였다.

동시에 개선이 필요한 문제도 확인하였다. 예를 들어 일부 작업에서는 코드 리뷰가 충분히 이뤄지기 전에 병합이 진행되었고, 브랜치 업로드 및 Pull Request 생성 주기가 일정하지 않았으며, Pull Request 생성 사실이 팀 전체에 즉시 공유되지 않는 경우가 있었다. 또한 데일리스크럼이 매일 일관되게 운영되지 않아 진행 상황 공유의 밀도가 낮아지는 시점도 존재하였다. 이러한 문제의 근본 원인은 초기 협업 프로세스가 아직 충분히 정착되지 않았고, 작업 우선순위에 대한 긴급성이 커지면서 검토 절차보다 구현과 병합이 앞서는 경우가 있었기 때문이다.

다음 스프린트에서는 이러한 회고 결과를 바탕으로 세 가지 개선 방향을 설정하였다. 첫째, 코드 리뷰를 일정 수준 이상 완료한 뒤 병합을 수행하여 품질 검토 절차를 강화한다. 둘째, 브랜치 분기와 Pull Request 생성을 더 짧은 주기로 수행하여 변경 사항을 자주 공유한다. 셋째, Pull Request 등록 시 Slack 또는 전체 공용 채널에 즉시 공지하고 데일리스크럼을 보다 규칙적으로 운영하여 협업 가시성을 높인다. 이와 같은 회고 기반 개선 활동은 개발 프로세스를 점차 정교하게 다듬는 데 중요한 역할을 하였다.

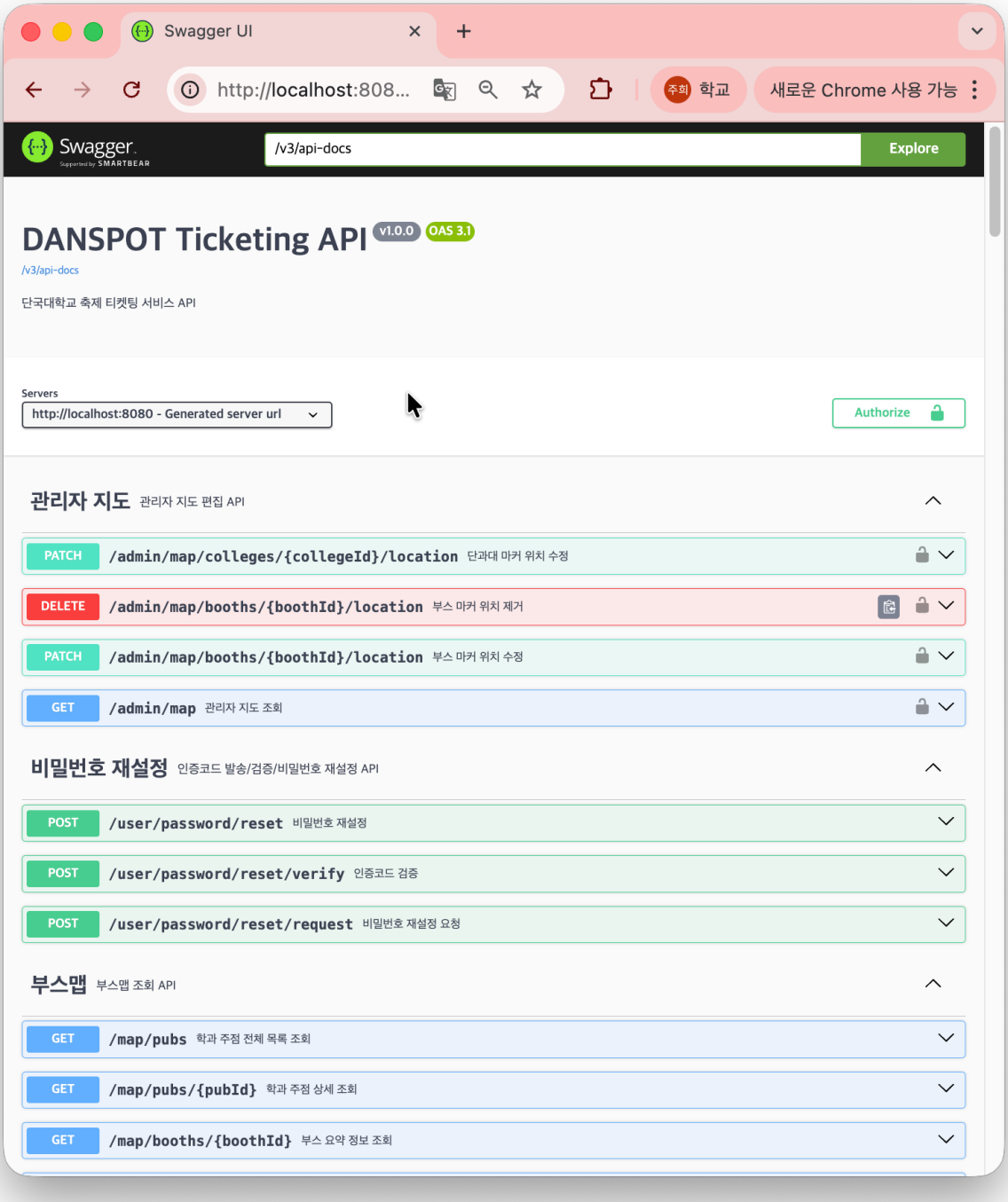
구분	사용 도구	활용 목적
버전 관리	GitHub	코드 관리, 브랜치 전략, Pull Request, 코드 리뷰
API 문서화	Swagger UI	API 명세 확인 및 요청·응답 테스트
일정 및 문서 관리	Notion	기획 문서, 회의록, 일정 정리
작업 관리	Jira	스프린트 기록, 작업 항목 관리
팀 소통	Slack, Discord	실시간 소통 및 개발 알림 공유
배포 및 테스트	Ngrok, NHN Cloud	외부 접속 테스트 및 테스트 서버 배포
모니터링 수집	Prometheus	백엔드 /actuator/prometheus, Redis Exporter, Prometheus 자체 메트릭 수집
대시보드 시각화	Grafana	부하 테스트 결과를 대시보드 형태로 시각화하고 주요 지표를 실시간으로 모니터링

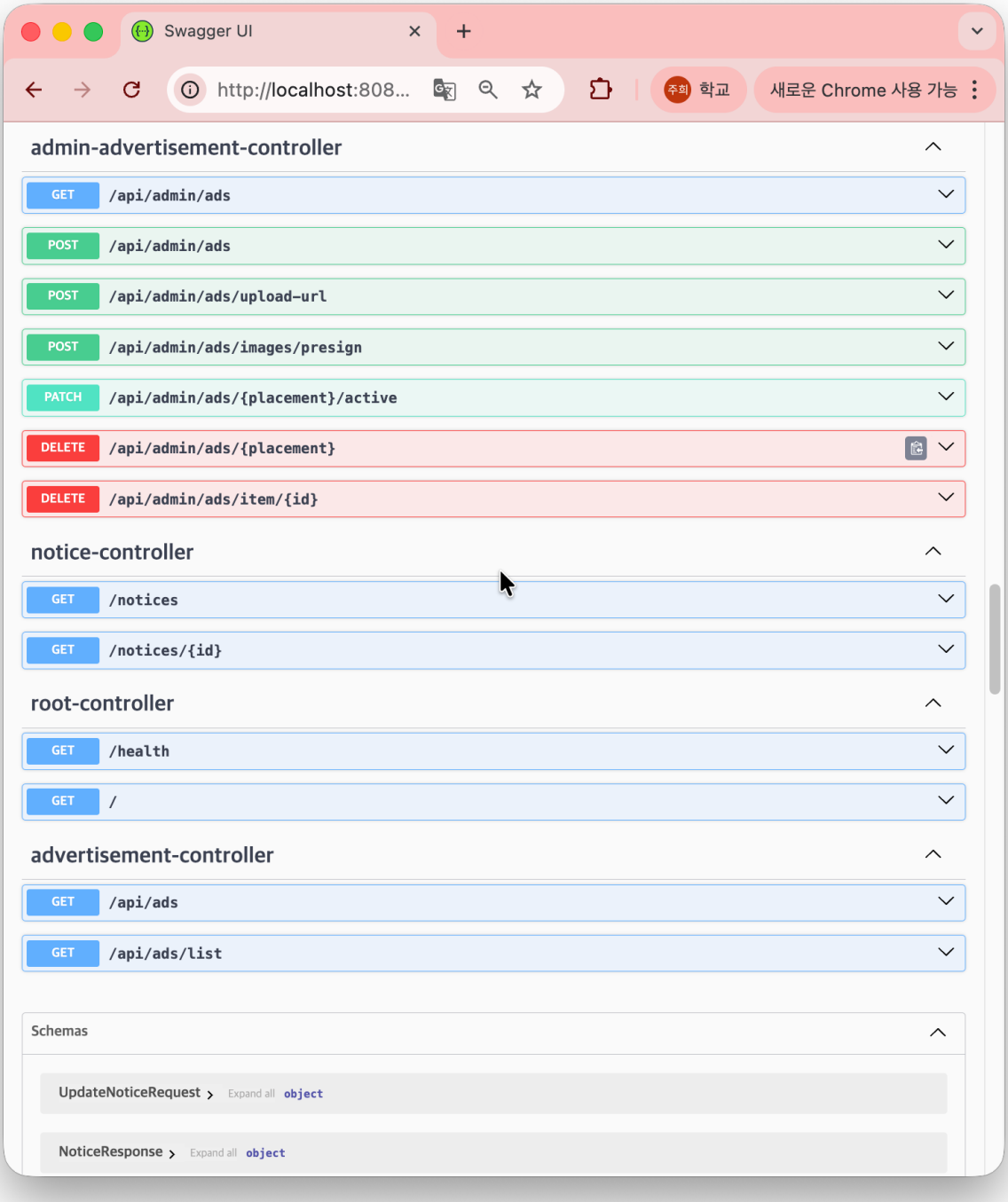
7. 개발 프로세스

feature 브랜치 생성 → 개발 및 커밋 → Pull Request 생성 → 코드 리뷰 및 CI 확인 → develop 병합 → main 배포

이와 같은 협업 환경을 통해 팀원별 역할을 분리하면서도 변경 사항을 체계적으로 관리할 수 있도록 하였다. 또한 API 문서화와 코드 리뷰 과정을 병행하여 프론트엔드와 백엔드 간 연동 오류를 줄이고 실제 배포 전 기능 안정성을 높이고자 하였다.

[Swagger UI / API 명세 화면]




Swagger UI
http://localhost:808...
학교
새로운 Chrome 사용 가능
admin-advertisement-controller
GET /api/admin/ads
POST /api/admin/ads
POST /api/admin/ads/upload-url
POST /api/admin/ads/images/presign
PATCH /api/admin/ads/{placement}/active
DELETE /api/admin/ads/{placement}
DELETE /api/admin/ads/item/{id}
notice-controller
GET /notices
GET /notices/{id}
root-controller
GET /health
GET /
advertisement-controller
GET /api/ads
GET /api/ads/list
Schemas
UpdateNoticeRequest Expand all object
NoticeResponse Expand all object

V. 진행 상황 및 향후 계획

1. 현재 진행 상황

현재 DANZZAN 은 서비스 기획과 설계를 바탕으로 핵심 기능 구현을 대부분 완료한 상태이며, 기능별 세부 화면과 일부 사용성 개선 작업을 진행하고 있다. 사용자 서비스에서는 홈, 타임테이블, 부스맵, 공지사항, 티켓팅, 내 티켓 기능을 구현하였고, 관리자 서비스에서는 공지 관리, 광고 배너 관리, 지도 관리, 티켓 조회 및 발급 처리 기능을 구현하였다.

티켓팅 기능의 경우 대기열 진입, 대기 상태 조회, 예매 처리, 내 티켓 조회와 같은 주요 흐름에 대한 테스트를 진행하였다. 이를 통해 다수의 사용자가 동시에 접근하는 상황에서 대기열 상태가 정상적으로 변경되는지, 예매 처리와 티켓 정보 저장이 의도한 흐름대로 이루어지는지 확인하였다.

현재는 서버 배포를 준비하고 있으며, 운영 환경에서 안정적으로 서비스가 동작할 수 있도록 설정 점검과 최종 테스트를 진행할 예정이다. 개발 단계에서는 Ngrok 을 활용하여 외부 접속 테스트를 수행하였고, 이후 NHN Cloud 기반 서버 배포를 통해 실제 사용자 접속이 가능한 환경으로 전환할 계획이다.

2. 배포 이후 운영 계획

DANZZAN 은 실제 축제 운영에 적용될 수 있는 서비스를 목표로 하기 때문에 배포 이후의 운영 계획도 함께 고려하였다. 배포 직후에는 사용자가 서비스를 이용하는 과정에서 발생하는 오류와 불편 사항을 빠르게 확인하고 주요 이슈에 대해서는 원인을 분석하여 수정할 예정이다.

특히 티켓팅 기능과 관리자 티켓 발급 기능은 축제 운영의 핵심 흐름과 직접 연결되므로 응답 속도, 대기열 처리 상태, 예매 성공 여부, 티켓 발급 처리 상태를 지속적으로 확인할 필요가 있다. 또한 실제 사용자 피드백을 바탕으로 UI/UX 를 개선하고 운영 과정에서 발견된 문제를 반영하여 서비스 안정성을 높일 계획이다.

구분	내용
초기 운영 모니터링	배포 직후 오류 및 사용자 불편 사항 확인
문제 해결 및 보완	운영 중 발생한 주요 이슈 분석 및 개선 적용
성능 지표 점검	응답 속도, 대기열 처리, 예매 흐름 분석
사용자 피드백 반영	실제 이용 의견 기반 UI/UX 개선

후속 기능 고도화	운영 결과를 반영한 기능 보완 및 서비스 향상

이와 같은 운영 계획을 통해 단순히 서비스를 배포하는 데 그치지 않고, 실제 사용 과정에서 발생하는 문제를 지속적으로 개선함으로써 안정적인 축제 운영을 지원하고자 한다.

3. 향후 개선 방향

향후 DANZZAN 은 대학 축제에 한정된 서비스에서 나아가 다양한 교내 행사에 적용 가능한 통합 행사 플랫폼으로 발전하는 것을 목표로 설정하였다. 현재 서비스는 회원 시스템, 공지 관리, 지도 정보, 티켓팅, 관리자 운영 기능을 포함하고 있기 때문에 축제 이후에도 체육대회, 학생회 행사, 설명회, 특강 등 다른 교내 행사에 확장 적용할 수 있다.

또한 서비스에 축적되는 공지사항, 타임테이블, 부스맵, 티켓 예매 정보 등을 활용하여 개인 맞춤형 안내 기능으로 확장할 수 있다. 예를 들어 사용자의 예매 티켓, 관심 시간대, 현재 위치를 바탕으로 축제 동선을 추천하거나, "오늘 2 시부터 6 시까지 어떻게 즐기면 좋을까?"와 같은 질문에 맞춤형 코스를 제안하는 개인화 AI 안내 기능을 후속 확장안으로 고려할 수 있다.

초기 단계에서는 축제 정보에 대한 질의응답 기능으로 시작하고, 이후에는 사용자의 예매 내역과 이용 흐름을 반영한 맞춤형 추천 기능으로 발전시킬 수 있다. 이를 통해 사용자가 여러 메뉴를 직접 탐색하지 않아도 필요한 정보를 자연어 기반으로 확인할 수 있으며, 축제 참여 경험의 편의성을 더욱 높일 수 있다.

개선 방향	내용
교내 행사 확장	축제 외 체육대회, 설명회, 학생회 행사 등으로 서비스 범위 확대
Personal AI 가이드	공지, 타임테이블, 부스맵, 티켓 정보를 기반으로 맞춤형 안내 제공
맞춤 동선 추천	예매 티켓, 관심 시간대, 위치 정보를 활용한 축제 코스 추천
운영 데이터 활용	티켓 예매, 공지 조회, 사용자 이용 흐름을 기반으로 향후 행사 기획에 활용
서비스 고도화	실제 운영 피드백을 반영한 기능 개선 및 안정화

결과적으로 DANZZAN 은 축제 기간에 필요한 정보를 제공하는 일회성 웹 서비스가 아니라, 실제 운영 경험과 사용자 데이터를 바탕으로 지속적으로 확장 가능한 교내 행사 플랫폼으로 발전할 가능성을 가진다.